



Arab Academy for Science, Technology, and Maritime Transport
College of Computing and Information Technology
Smart Village

Cerberus Platform

Graduation Project

A Thesis submitted in partial fulfillment of the requirements of
B.Sc. in “Cybersecurity”

by	
Abdelrahman Mohamed Mahmoud	221027770
Nour Mohamed Mahmoud	221027637
Karim Osama Abdelmoniem	221027651
Mohamed Tamer Eid	221007879
Mohamed Hossam Mohamed	221007946
Omar Hisham Hamed	221027535

College of Computing and Information Technology, AASTMT

Supervised By
Assoc.Prof. Mohamed Hussien

Smart Village, Cairo, 2025-2026

Abstract

Modern cyber threats — ranging from ransomware to advanced persistent threats (APTs) — demand a fundamentally different defensive posture from traditional signature-based or perimeter-centric security approaches. The Cerberus Platform addresses this demand by combining two powerful defensive primitives: deception-based early-warning and automated behavioral sandboxing. Cerberus deploys a mesh of carefully crafted canary files across monitored endpoints; any interaction with these files triggers an immediate, high-fidelity alert that bypasses the probabilistic false-positive burden endemic to anomaly detectors. Simultaneously, a lightweight Python endpoint agent monitors running processes and file system events, captures forensic artefacts, and submits suspicious binaries asynchronously to a CAPEv2 malware analysis sandbox. The resulting behavioural report is enriched via a two-phase VirusTotal API pipeline, correlated against 40 MITRE ATT&CK techniques, and presented in real-time on a WebSocket-driven SOC dashboard. Automated containment responses — process termination, network isolation, and evidence preservation — are dispatched within ten seconds of confirmed threat classification. Empirical evaluation over ninety test runs using real-world ransomware, credential-stealer, and lateral-movement payloads demonstrated a 97.8% canary-triggered detection rate, a mean pipeline-closure time of 8.4 seconds, an endpoint CPU overhead below 0.8%, and zero false-positive incidents after enabling the five-tier suppression engine. This document constitutes the complete Software Requirements Specification (IEEE 830), system design, implementation, and testing record for the Cerberus Platform.

Keywords: deception technology, canary files, malware sandboxing, CAPEv2, MITRE ATT&CK, endpoint detection and response, incident response automation, behavioral analysis.

Acknowledgment

The authors wish to express their sincere gratitude to the supervisory and academic staff of the Arab Academy for Science, Technology, and Maritime Transport for their continuous guidance throughout this project. Special thanks are extended to Dr. Mohamed Hussein whose expertise in cybersecurity and systems architecture shaped the technical direction of the Cerberus Platform. We are also grateful to the open-source community — particularly the CAPEv2, Volatility, and YARA maintainers — whose tools form the analytical backbone of this system. Finally, we thank our families for their unwavering support and encouragement during the long hours of development and testing.

Table of Contents

CONTENTS	PAGE
Abstract	ii
Acknowledgment	iii
List of Figures	v
List of Tables	vi
Abbreviations	vii
Symbols	ix
Chapter 1 – Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Problem Complexity	4
1.5 Constraints	6
1.6 Standards	9
1.7 Feasibility Study and Business Canvas	11
1.8 Thesis Organization	18
Chapter 2 – Background	19
2.1 Introduction of modern cybersecurity challenges	19
2.2 Deception Technology Foundations	19
2.3 Automated Malware Analysis	20
2.4 Incident Response and automation	21
2.5 Integration Gaps in current Security Solutions	22
2.6 Theoretical Foundations	23
2.7 Related Academic Work	23
Chapter 3 – Related Work and Similar Systems	25
3.1 Related Work	25
3.2 Similar Systems	28
3.3 Research Gaps and Cerberus Innovation	32
Chapter 4 – Analysis (SRS, IEEE 830)	34
4.1 Functional Requirements	34
4.2 Non-Functional Requirements	37
Chapter 5 – Design	39
5.1 Design Alternatives	39
5.2 UML Diagrams	41
5.3 Technologies and Tools Used	45
5.4 Prototype	47

Chapter 6 – Implementation	52
6.1 Implementation Environment	52
6.2 Platform Architecture and Subsystem Implementation	53
6.3 Results and Discussion	55
Chapter 7 – Testing (STP, IEEE 829)	56
7.1 Introduction and Test Objectives	56
7.2 Test Items and Features to be Tested	56
7.3 Test Matrix (IEEE 829)	56
7.14 Test Deliverables	57
Chapter 8 – Conclusion and Future Work	60
References	63
Appendix A – Senior Project Summary Report	65

List of Figures

FIGURE	CAPTION	PAGE
Figure 2.7.8	Business Model Canvas	17
Figure 5.2.1	Cerberus Platform Use Case Diagram	41
Figure 5.2.2	Cerberus Platform Class Diagram	41
Figure 5.2.3	End-to-End Attack Detection & Response Sequence Diagram	42
Figure 5.2.4	Data Flow Diagram (DFD) and Threat Model	43
Figure 5.2.5	Cerberus Modular Components	44
Figure 5.3	Technologies and Tools Stack	45
Figure 5.4.1	SOC Dashboard Overview	47
Figure 5.4.1	Active Incidents Interface	47
Figure 5.4.2	Malware Sandbox Analysis Interface	48
Figure 5.4.2	Extracted Indicators of Compromise (IOCs) Panel	48
Figure 5.4.3	Security Report Generator	49
Figure 5.4.4	Deception Canary Management Dashboard	49
Figure 5.4.5	Agent Health and Activity Logs	50
Figure 5.4.5	Real-time Agent Logs View	50
Figure 5.4.6	Endpoint Activity Monitoring (File, Process, Network)	51

List of Tables

TABLE	CAPTION	PAGE
Table 1.5	Project Constraints and Acceptance Criteria	6
Table 1.6	Applied Standards and Verification Methods	9
Table 2.7.2.1	Core Technologies and Feasibility Rationale	11
Table 2.7.2.2	System Integration Feasibility Analysis	12
Table 2.7.4.1	Development Cost Estimation	13
Table 2.7.4.3	Return on Investment (Academic Perspective)	13
Table 2.7.5.3	Project Milestone Schedule	14
Table 2.7.6.1	Licensing Considerations and Compliance	15
Table 2.7.7.1	Identified Risks and Mitigation Strategies	15
Table 2.7.8.1	Overall Feasibility Assessment	16
Table 3.2.4	Commercial Solutions Benchmark Comparison	31
Table 5.3	Detailed Technology Stack and Libraries	46
Table 6.1:	Virtual Machine Configuration	52
Table 6.2:	Software Stack with Version Pins	52
Table 6.3:	Canary File Archetypes and Placement	54
Table 6.4:	False-Positive Suppression Mechanisms	55
Table 6.5:	CAPE_TO_MITRE_MAP — 40-Entry Technique Map	58
Table 6.6:	VirusTotal Enrichment Field Mapping	61
Table 6.7:	REST API Endpoint Specification	62
Table 6.8:	Database ORM Model Field Definitions	64
Table 6.9:	Detection Accuracy Results by Category	65
Table 6.10:	System Performance Benchmark Summary	66
Table 6.11:	Implementation Challenges and Resolutions	66

Abbreviations

Abbreviation	Meaning
AES	Advanced Encryption Standard
API	Application Programming Interface
APT	Advanced Persistent Threat
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
BMC	Business Model Canvas
C2	Command and Control
CAC	Customer Acquisition Cost
CEF	Common Event Format
CPU	Central Processing Unit
CSV	Comma-Separated Values
DLP	Data Loss Prevention
DNS	Domain Name System
ECC	Elliptic Curve Cryptography
EDR	Endpoint Detection and Response
GDPR	General Data Protection Regulation
HTTP	Hypertext Transfer Protocol
IEC	International Electrotechnical Commission
IEEE	Institute of Electrical and Electronics Engineers
IOC	Indicator of Compromise
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LOLBins	Living-off-the-land Binaries
LTV	Lifetime Value
MITM	Man-in-the-Middle

Abbreviation	Meaning
MSSP	Managed Security Service Provider
MTTD	Mean Time to Detect
MTTR	Mean Time to Respond
NIST	National Institute of Standards and Technology
OWASP	Open Web Application Security Project
PDF	Portable Document Format
PII	Personally Identifiable Information
RAM	Random Access Memory
ROI	Return on Investment
RSA	Rivest–Shamir–Adleman (Encryption)
SIEM	Security Information and Event Management
SMB	Small/Medium Business (Business context) / Server Message Block (Protocol)
SOAR	Security Orchestration, Automation, and Response
SOC	Security Operations Center
SQL	Structured Query Language
SQuaRE	Systems and Software Quality Requirements and Evaluation
SRS	Software Requirements Specification
SSH	Secure Shell
STIX	Structured Threat Information eXpression
TAXII	Trusted Automated Exchange of Intelligence Information
TLS	Transport Layer Security
TTP	Tactics, Techniques, and Procedures
UEBA	User and Entity Behavior Analytics
UML	Unified Modeling Language
URL	Uniform Resource Locator
VAR	Value-Added Reseller
VM	Virtual Machine

Symbols

Symbol	Description
\$	United States Dollar (Currency)
%	Percentage (Ratio/Rate)
<	Less than
>	Greater than
≤	Less than or equal to
≥	Greater than or equal to

Chapter 1 - Introduction

1.1 Motivation

The **cybersecurity landscape** has undergone a dramatic transformation over the past decade, with attackers employing increasingly sophisticated techniques that render traditional defensive mechanisms insufficient. Organizations face an escalating threat from **polymorphic malware, zero-day exploits, and fileless attacks** that evade signature-based detection. The **2023 Verizon Data Breach Investigations Report** indicates that **74% of breaches** involved human elements, and the average time to identify a breach remains **207 days**, providing adversaries extensive windows for reconnaissance and data exfiltration.

Current enterprise security architectures exhibit **three critical deficiencies**:

1. **Reactive Nature:** EDR solutions primarily operate in reactive mode, detecting threats only after initial compromise.
2. **Operational Silos:** Malware analysis workflows require manual submission to sandboxes, introducing delays of hours or days.
3. **Alert Fatigue:** Insider threats and lateral movement remain difficult to detect without generating overwhelming false positive rates.

Deception technology represents a paradigm shift toward proactive defense, establishing high-fidelity tripwires with **near-zero false positives**. However, existing solutions operate in isolation, requiring manual correlation. Similarly, behavioral sandboxing lacks integration with real-time detection mechanisms.

The motivation for Cerberus emerges from bridging these gaps to create an **integrated platform** where deception, analysis, and response function as a cohesive strategy. By automatically channeling detected threats through behavioral analysis and orchestrating containment, Cerberus promises to dramatically reduce both **Mean Time to Detect (MTTD)** and **Mean Time to Respond (MTTR)**.

Furthermore, the academic community lacks **open, well-documented implementations** of such integrated architectures. Cerberus aims to contribute to the literature by documenting the design decisions and performance metrics of integrating deception with automated analysis.

1.2 Problem Statement

Modern enterprises confront a multi-faceted security challenge that existing solutions inadequately address. Specifically:

1.2.1 Problem 1: Detection Lag and Reactive Posture

Traditional security controls detect threats only after execution, when malware has already begun encryption (ransomware), data collection (spyware), or credential harvesting (stealers). This reactive posture allows attackers critical time to achieve their objectives before security teams can respond.

1.2.2 Problem 2: Manual Analysis Bottleneck

When suspicious files are identified, analysts must manually submit them to sandbox environments, wait for analysis completion (typically 10-20 minutes), review behavioral reports, extract indicators of compromise (IOCs), and then initiate response actions. This serialized, human-dependent workflow introduces delays measured in hours, during which the threat may propagate across the network.

1.2.3 Problem 3: Insider Threat Detection Gaps

Malicious insiders and compromised credentials enable lateral movement that traditional perimeter defenses cannot detect. Behavior analytics generate numerous false positives when attempting to identify anomalous access patterns, overwhelming security teams and degrading detection efficacy.

1.2.4 Problem 4: Lack of Integrated Defense

Security infrastructure consists of disconnected point solutions: deception systems generate alerts, EDR platforms detect endpoint anomalies, sandboxes analyze samples, and SIEM platforms aggregate logs. Security analysts must manually correlate data across these systems, reconstruct attack timelines, and coordinate response actions a process prone to errors and delays.

1.2.5 Problem 5: Limited Forensic Context

When alerts trigger, analysts often lack sufficient forensic context to prioritize response. Was the triggered canary file accessed deliberately or incidentally? What process initiated the access? What network connections were established? What other systems has the compromised host contacted? Answering these questions requires time-consuming investigation across multiple tools.

1.2.6 Research Question:

Can an integrated platform combining strategic deception, automated behavioral analysis, and orchestrated incident response significantly reduce threat detection and response times while maintaining acceptable false positive rates and providing actionable forensic intelligence?

The Cerberus project addresses this problem statement by architecting a unified platform where detection immediately triggers analysis, analysis informs automated response, and response actions preserve comprehensive forensic evidence transforming defensive operations from reactive and fragmented to proactive and coordinated.

1.3 Objectives

1.3.1 Primary Objective

Develop and deploy an advanced, Multi-Threat Deception and Analysis Platform that combines behavioral sandboxing, canary file deception, and intelligent incident response to proactively defend endpoints and critical assets against evolving malware threats including ransomware, data stealers, and insider attacks.

1.3.2 Secondary Objectives

1.3.2.1 Real-Time Threat Detection: Deliver real-time threat detection using dynamically monitored honeypots (canary files and tokens) with behavioral analysis that achieves High true positive detection accuracy while maintaining Low false positive rates.

1.3.2.2 Automated Response Orchestration: Orchestrate automated incident response workflows that minimize mean time to detect (MTTD) and mean time to respond (MTTR) when threats are identified, implementing containment actions without human intervention.

1.3.2.3 Centralized Security Operations: Provide a centralized web-based dashboard offering security teams actionable visibility into threat landscape, comprehensive forensic data, and granular system control with intuitive interfaces that enable effective triage and investigation.

1.3.2.4 Cross-Platform Deployment: Ensure modularity, scalability, and cross-platform deployment capabilities supporting Windows 10/11 and Ubuntu 20.04+ environments with the ability to monitor 50-200 endpoints in academic proof-of-concept scenarios.

1.3.2.5 Contextual Threat Intelligence: Reduce false positives through contextual signal correlation across file access patterns, process behavior, network connections, and user activity, enriching alerts with actionable threat intelligence mapped to MITRE ATT&CK frameworks.

1.3.3 Research Objectives

1.3.3.1 Methodology Advancement: Advance cybersecurity research by evaluating the effectiveness of combining sandboxing with canary deception in an integrated architecture, documenting quantifiable improvements in detection accuracy and response times.

1.3.3.2 Implementation Guidance: Document practical implementation challenges, architectural decisions, and configuration recommendations for deploying deception-based detection in enterprise environments.

1.3.3.3 Security Metrics: Establish measurable security outcomes and key performance indicators (KPIs) for deception-integrated defense platforms, enabling evidence-based comparison with traditional security architectures.

1.3.3.4 Academic Contribution: Produce publishable research documenting the Cerberus architecture, performance benchmarks, and operational lessons learned, contributing to the academic body of knowledge in proactive cyber defense.

1.4 Problem Complexity

1.4.1 Wide-Ranging and Conflicting Technical Issues

The platform must reconcile competing requirements across multiple technical domains:

- **Performance vs. Security Trade-offs:** Continuous file system monitoring and behavioral analysis impose computational overhead that must be minimized to avoid degrading endpoint performance, yet comprehensive monitoring is essential for detection efficacy.
- **Detection Accuracy vs. False Positive Balance:** Achieving high detection rates while maintaining operationally acceptable false positive rates requires sophisticated signal correlation and behavioral baselining, particularly for insider threat detection.

1.4.2 No Obvious Solution

Unlike established problems with well-known solutions, integrating deception with automated analysis presents novel challenges:

- **Deception Detection:** Sophisticated malware may attempt to identify canary files through pattern recognition or environmental fingerprinting, requiring dynamic deception strategies and anti-detection techniques.
- **Automated Decision-Making:** Determining when to execute containment actions without human authorization requires confidence thresholds, risk assessment algorithms, and fail-safe mechanisms to prevent operational disruption from false positives.

1.4.3 Multiple Disciplines

Successful implementation requires expertise spanning:

- **Cybersecurity:** Threat modeling, malware analysis, forensics, intrusion detection, incident response
- **Systems Engineering:** Distributed architectures, inter-process communication, fault tolerance, scalability
- **Software Development:** Backend services, API design, database management, frontend interfaces
- **Network Security:** Traffic analysis, network isolation, packet inspection
- **Data Science:** Behavioral analytics, anomaly detection, machine learning (for future enhancements)

1.4.4 Diverse Stakeholders

Successful implementation requires expertise spanning:

- **Security Operations Center (SOC) Analysts:** Require intuitive interfaces, actionable alerts, comprehensive forensic data, and rapid triage capabilities
- **Incident Response Teams:** Need detailed technical evidence, chain-of-custody documentation, and integration with existing response workflows
- **IT Operations:** Demand minimal performance impact, easy deployment, reliable operation, and minimal false positives
- **Compliance Officers:** Require audit trails, evidence preservation, regulatory compliance (GDPR)

- **Executive Management:** Seek measurable risk reduction, cost-effectiveness, and operational efficiency improvements

1.4.5 Involves Multiple Component Parts

The architecture comprises interdependent subsystems, each presenting engineering challenges:

1. **Endpoint Agent Module:** Requires privilege escalation, tamper resistance, reliable communication under adverse conditions
2. **Deception Management:** Demands strategic canary placement, file integrity verification, credential lifecycle management
3. **Sandbox Integration:** Involves VM orchestration, sample collection, result parsing, failure handling
4. **Analysis Correlation Engine:** Necessitates event aggregation, behavioral scoring, threshold tuning
5. **Response Orchestration:** Requires action sequencing, rollback capabilities, evidence preservation
6. **Central Management Server:** Demands concurrent connection handling, data persistence, API security
7. **Dashboard Interface:** Needs real-time data streaming, responsive design, role-based access control

1.5 Constraints

Table 1.5: Project Constraints and Acceptance Criteria

Constraint Category	Specific Constraint	Measurement Metric	Verification Method	Acceptance Criteria	Effect on Cerberus
Performance	Agent CPU Utilization	CPU percentage (%)	Performance Monitor (Windows), top/htop (Linux)	≤ 8% average, ≤ 15% peak during scanning	Maintains agent stealth and prevents interference with normal user activities.
Performance	Agent Memory Footprint	RAM consumption (MB)	Task Manager (Windows), free/htop (Linux)	≤ 150MB baseline, ≤ 250MB during analysis	Ensures cross-platform compatibility on resource-constrained enterprise endpoints.
Performance	Detection Latency	Time from access to alert (seconds)	Timestamp logging, Prometheus monitoring	≤ 10 seconds for canary access detection	Minimizes the window for ransomware encryption by accelerating detection
Performance	Analysis Throughput	Concurrent sandbox analyses	CAPEv2 metrics, system monitoring	3-8 simultaneous analyses without degradation	Defines the system's capacity to process simultaneous threats or malware outbreaks.
Performance	Dashboard Response Time	Page load time (seconds)	Browser DevTools, Lighthouse	≤ 3 seconds for dashboard initial load	Maximizes SOC analyst efficiency and situational awareness during incident triage.
Reliability	System Uptime	Availability percentage (%)	Grafana uptime monitoring, health checks	≥ 95% in lab environment (excluding maintenance)	Eliminates "blind spots" to ensure continuous monitoring of critical assets
Reliability	Agent Crash Recovery	Time to automatic restart (seconds)	Watchdog service logs, system journal	≤ 90 seconds from crash to resumed monitoring	Limits the timeframe an attacker can operate undetected after killing the agent process.
Reliability	Data Persistence During Outage	Queued alerts retained	Queue depth monitoring, alert database	≥ 95% of alerts queued for up to 48 hours	Prevents the loss of forensic evidence during network or server outages.
Security	Communication Encryption	Protocol version	Network traffic analysis (Wireshark)	TLS 1.2 or higher for all agent-server traffic	Secures incident data and command signals against interception or MITM attacks
Security	Authentication Mechanism	Token validation success rate (%)	Authentication logs, failed login tracking	100% requests require valid API tokens	Prevents rogue agents or unauthorized users from flooding or accessing the platform.
Security	Data Encryption at Rest	Encryption algorithm	Database configuration audit, file	AES-256 for all stored sensitive data	Safeguards forensic repositories and sensitive data against database breaches

<i>Constraint Category</i>	<i>Specific Constraint</i>	<i>Measurement Metric</i>	<i>Verification Method</i>	<i>Acceptance Criteria</i>	<i>Effect on Cerberus</i>
Scalability	Endpoint Support	Number of concurrent agents	Load testing, connection pool monitoring	50-200 endpoints without performance degradation	Sets the operational boundary for the proof-of-concept server stability.
Scalability	Alert Storage Capacity	Total alerts retained	Database size monitoring, query performance	10,000-50,000 alerts with query times ≤ 2 seconds	Balances forensic history depth with real-time database query performance.
Scalability	Daily Analysis Capacity	Samples analyzed per day	Sandbox queue metrics, completion logs	20-100 malware samples processed to completion	Establishes the operational limit for sandbox hardware resource utilization
Usability	Dashboard Learning Curve	Time to basic proficiency (minutes)	User testing sessions, task completion tracking	≤ 30 minutes for SOC analysts to perform triage	Validates system design by ensuring rapid adoption for new security analysts.
Usability	Alert Triage Time	Time per incident (minutes)	User testing, workflow timing	≤ 5 minutes to assess and assign incident	Directly reduces the platform's overall Mean Time to Respond (MTTR).
Usability	Report Generation Time	Report completion (seconds)	System logs, user feedback	≤ 30 seconds for standard incident report	Facilitates faster delivery of forensic reports for legal and management review
Compatibility	Operating System Support	Platform versions	Installation testing, agent deployment	Windows 10/11 (x64), Ubuntu 20.04+ (x64) verified	Guarantees consistent behavioral monitoring across both Windows and Linux environments.
Compatibility	Browser Support	Browser versions	Cross-browser testing	Chrome 90+, Firefox 85+ fully functional	Ensures dashboard accessibility for SOC teams using standard modern web tools
Compatibility	Python Version	Language version	Dependency checking, runtime verification	Python 3.8-3.11 compatibility confirmed	Maintains stability and compatibility with open-source backend dependencies
Deployment	Agent Installation Time	Installation duration (minutes)	Deployment scripts, timing logs	≤ 10 minutes per endpoint (automated script)	Minimizes the administrative overhead and time required for enterprise rollout
Deployment	Server Setup Time	Complete deployment (hours)	Installation documentation, deployment logs	≤ 4 hours with Docker (including CAPEv2 config)	Enables rapid platform recovery or deployment via containerized environments.
Cost	Infrastructure Budget	Total hardware, software cost	Budget, license costs	$\leq \$5,000$ for proof-of-concept (academic licenses)	Adheres to academic constraints by prioritizing free and open-source software.

<i>Constraint Category</i>	<i>Specific Constraint</i>	<i>Measurement Metric</i>	<i>Verification Method</i>	<i>Acceptance Criteria</i>	<i>Effect on Cerberus</i>
Legal	Data Retention Period	Storage duration (days)	Configuration audit, automated cleanup logs	30-90 days configurable based on storage capacity	Ensures compliance with data privacy laws while maintaining forensic history.
Legal	Privacy Compliance	Regulation adherence	Privacy impact assessment, data flow audit	GDPR-compliant data handling with anonymization	Mitigates legal risks by implementing anonymization in endpoint monitoring
Time	Project Completion	Total development time (weeks)	Project schedule, milestone tracking	30 weeks (Project 1: 15 weeks, Project 2: 15 weeks)	Defines the prototype timeline to ensure all core goals are met for defense.
Detection Accuracy	True Positive Rate	Detection accuracy (%)	Confusion matrix from test dataset	$\geq 85\%$ for ransomware, $\geq 80\%$ overall	Measures the effectiveness of combining behavioral analysis with deception.
Detection Accuracy	False Positive Rate	FP percentage (%)	Alert validation, analyst feedback	$\leq 8\%$ in controlled testing environment	Prevents alert fatigue to ensure analysts trust and act on system warnings.
Detection Accuracy	False Negative Rate	FN percentage (%)	Missed detection tracking, penetration testing	$\leq 18\%$ (acceptable for prototype scope)	Sets a realistic baseline for protection against sophisticated evasion techniques.

1.6 Standard

Table 1.6: Applied Standards and Verification Methods

Standard	Full Name	Application in Cerberus	Document Reference	Verification Method
IEEE 12207	Software Life Cycle Processes	Overall project methodology, process documentation	All chapters	Process audit, milestone reviews
IEEE 830	Software Requirements Specification	Requirements documentation structure	Chapter 4	Requirements traceability matrix
IEEE 829	Software Test Plan	Testing methodology and documentation	Chapter 7	Test case execution logs
ISO/IEC 25010	Systems and Software Quality Requirements and Evaluation (SQuaRE)	Quality attributes and metrics definition	Chapter 4 (NFRs), Chapter 7	Quality metric measurements
ISO/IEC 27001	Information Security Management	Security controls and audit logging	Chapter 4, Chapter 6	Security audit, compliance checklist
NIST Cybersecurity Framework	Risk management and security controls	Threat modeling, security architecture	Chapter 4 (Section 4.2.6)	Framework mapping document
OWASP Top 10	Web application security vulnerabilities	Dashboard security implementation	Chapter 5, Chapter 6	OWASP ZAP scanning, code review
MITRE ATT&CK	Adversarial tactics and techniques framework	Threat intelligence mapping and IOC categorization	Chapter 4, Chapter 5	ATT&CK technique mapping in reports
UML 2.5	Unified Modeling Language	System design and architecture diagrams	Chapter 4, Chapter 5	Diagram validation against spec
RESTful API Design	Representational State Transfer	API architecture between components	Chapter 5, Chapter 6	API documentation, Postman testing
GDPR	General Data Protection Regulation	Data handling, privacy design, retention policies	Chapter 4 (Section 4.2.2)	Privacy impact assessment

Standard	Full Name	Application in Cerberus	Document Reference	Verification Method
JSON Schema	JSON data validation	Configuration files and API payloads	Chapter 6	Schema validation in unit tests
OpenAPI 3.0	API documentation standard	Cerberus API specification	Chapter 5	Swagger UI generation
STIX 2.1	Structured Threat Information eXpression	Threat intelligence export format	Chapter 4, Chapter 6	STIX validation tools
Common Event Format (CEF)	Log and event data format	SIEM integration compatibility	Chapter 4, Chapter 6	CEF parser validation
PEP 8	Python code style guide	Python code formatting and conventions	Chapter 6	Pylint, Black code formatter
Python Package Index (PyPI)	Python dependency management	Dependency specification and pinning	Chapter 5, Chapter 6	requirements.txt verification

1.7 Feasibility Study and Business Canvas

1.7.1 Feasibility Study

1.7.1.1 Project Overview

Cerberus is an integrated cybersecurity platform combining deception technology with automated malware analysis. The system detects threats through strategically placed canary files, automatically submits suspicious artifacts for behavioral analysis, and provides automated response capabilities. This study evaluates the technical, operational, economic, and schedule feasibility of developing Cerberus as a graduation project.

1.7.1.2 Key Findings

- ☐ **Technical Feasibility:** High - Components are proven, integration path is clear
- ☐ **Operational Feasibility:** Moderate - Requires careful performance optimization
- ☐ **Economic Feasibility:** High - Uses open-source tools, minimal costs
- ☐ **Schedule Feasibility:** High - 20-week timeline is achievable with defined milestones
- ☐ **Legal/Regulatory Feasibility:** High - All components are open-source, no licensing issues

1.7.2 Technical Feasibility

1.7.2.1 Technology Stack Analysis

Core Technologies:

Table 2.7.2.1: Core Technologies and Feasibility Rationale

Component	Technology	Feasibility	Rationale
Detection Agent	Python + watchdog	HIGH	Mature libraries, cross-platform
Web Dashboard	Flask + Bootstrap	HIGH	Lightweight, rapid development
Sandbox Integration	Sandbox	HIGH	Well-documented REST API
Database	SQLite/PostgreSQL	HIGH	SQLAlchemy provides abstraction
Communication	REST API + TLS	HIGH	Standard web technologies

Critical Technical Challenges:

1. **File System Performance:** Monitoring must not impact endpoint performance (<8% CPU target)
2. **Real-time Processing:** Detection-to-alert pipeline must complete within 10 seconds
3. **Sandbox Integration:** Asynchronous job management and result parsing
4. **Cross-platform Support:** Windows/Linux agent compatibility

Mitigation Strategies:

- Use efficient file monitoring (inotify/ReadDirectoryChangesW)
- Implement queue-based processing for scalability
- Use Docker containers for sandbox isolation
- Abstract platform-specific code with clear interfaces

1.7.2.2 Integration Feasibility

Table 2.7.2.2: System Integration Feasibility Analysis

Integration Point	Complexity	Risk Level	Mitigation
Sandbox API	Low	Low	Direct REST API, documented endpoints
File System Hooks	Medium	Medium	Platform-specific implementations
Network Communication	Low	Low	Standard HTTPS with TLS
Dashboard Updates	Low	Low	WebSocket/Server-Sent Events

1.7.2.3 Performance Feasibility

Acceptance Criteria vs. Expected Performance:

NFR-PERF-001 (CPU ≤8%):

Expected: 3-8% (Python efficiency + optimized monitoring)

Risk: Medium - Requires careful implementation

NFR-PERF-002 (Detection Latency ≤10s):

Expected: 5-8 seconds (LAN conditions)

Risk: Low - Simple event pipeline

NFR-SCAL-001 (50-200 endpoints):

Expected: 100 endpoints comfortable load

Risk: Low - Stateless architecture scales

1.7.3 Operational Feasibility

1.7.3.1 Development and Deployment

Development Environment:

- ☐ **Languages:** Python (95%), JavaScript (5%)
- ☐ **Tools:** Git, Docker, VS Code/PyCharm
- ☐ **Team Size:** 1 developer (student) with advisor support
- ☐ **Documentation:** Comprehensive README, API docs, setup guides

Deployment Requirements:

Minimum Requirements:

Development: 8GB RAM, 4-core CPU, 100GB storage

Production (Demo): 16GB RAM, 8-core CPU, 200GB storage

Agents: 2GB RAM, 2-core CPU per endpoint

Software Dependencies:

Python 3.8+, Docker 20+, VirtualBox 6+ (for CAPEv2)

Windows 10/11 or Ubuntu 20.04+

1.7.3.2 Maintenance and Support

Long-term Considerations:

- ☐ **Update Strategy:** Docker containers for easy updates
- ☐ **Monitoring:** Built-in health checks and metrics
- ☐ **Backup:** Automated database backups
- ☐ **Security:** Regular dependency updates, security patches

Support Requirements:

- **During Development:** GitHub issues, advisor meetings
- **Post-Completion:** Documentation, demo scripts
- **Community Support:** Open-source model encourages contributions

1.7.4 Economic Feasibility

1.7.4.1 Development Costs

Table 2.7.4.1: Development Cost Estimation

Cost Category	Estimated Cost	Justification
Hardware	\$0	University lab equipment available
Software	\$0	All open-source tools
Cloud Services	\$0	Local development only
Licensing	\$0	Apache 2.0 for code, open-source dependencies
Contingency	\$200	Unexpected software/tools
Total	\$200	Minimal out-of-pocket costs

1.7.4.2 Resource Requirements

Human Resources:

- ☐ **Primary Developer:** 20 hours/week × 20 weeks = 400 hours
- ☐ **Academic Advisor:** 2 hours/week × 20 weeks = 40 hours
- ☐ **Testing/QA:** 4 hours/week × 20 weeks = 80 hours
- ☐ **Documentation:** 3 hours/week × 20 weeks = 60 hours
- ☐ **Total: 580 person-hours**

Infrastructure Resources:

- ☐ **Development Machine:** Provided by university
- ☐ **Test Endpoints:** 2–3 physical/virtual machines
- ☐ **Network:** University lab network
- ☐ **Storage:** University-provided storage

1.7.4.3 Return on Investment (Academic Perspective)

Table 1.7.4.3: Return on Investment (Academic Perspective)

Benefit Category	Value
Academic Credit	Graduation requirement fulfillment
Portfolio Development	Showcase project for employment
Research Publication	Potential conference/journal paper
Skill Development	Full-stack security engineering
Industry Relevance	Addresses current cybersecurity needs
Networking	Conference presentations, GitHub visibility

Cost-Benefit Ratio: Highly favorable - Significant academic and career benefits for minimal financial investment.

1.7.5 Schedule Feasibility

5.1 Project Timeline (20 Weeks)

Section Phase 1: Foundation (Weeks 1-4)

Literature Review & Research :2026-01-12, 10d
 Requirements Specification :2026-01-22, 81d
 Architecture Design :2026-01-30, 10d
 Environment Setup :2026-02-02, 6d

Section Phase 2: Core Development (Weeks 5-10)

Detection Agent Development :2026-02-09, 21d
Central Server API :2026-02-16, 21d
CAPEv2 Setup :2026-02-23, 14d
Basic Dashboard :2026-03-02, 14d

Section Phase 3: Integration (Weeks 11-14)

Sandbox Integration :2026-03-09, 14d
Automated Forensic Collection :2026-03-16, 14d
Multi-threat Detection Logic :2026-03-23, 14d
Alerting System :2026-03-23, 10d

Section Phase 4: Advanced Features (Weeks 15-16)

Automated Response Engine :2026-04-06, 14d
Enhanced Dashboard :2026-04-13, 14d
Performance Optimization :2026-04-20, 10d

Section Phase 5: Testing & Polish (Weeks 17-18)

Unit & Integration Testing :2026-04-27, 14d
Security Testing :2026-05-04, 10d
Performance Validation :2026-05-08, 7d

section Phase 6: Finalization (Weeks 19-20)

Documentation :2026-05-15, 10d
Demo Preparation :2026-05-18, 7d
Final Presentation Prep :2026-05-22, 5d
Thesis Submission :2026-05-27, 3d

1.7.5.2 Critical Path Analysis

Critical Path Components:

1. **Detection Agent** (Weeks 5–8) - Foundation of entire system
2. **Sandbox Integration** (Weeks 9–12) - Core innovation point
3. **Dashboard** (Weeks 9–12) - Required for demonstration

Buffer Time: 4 weeks allocated for unexpected delays

Risk Mitigation: Modular design allows parallel development

1.7.5.3 Milestone Schedule

Table 2.7.5.3: Project Milestone Schedule

Milestone	Week	Success Criteria
M1: Research Complete	3	Literature review, requirements defined
M2: Agent Prototype	8	Basic file monitoring working
M3: Core Integration	12	Detection → Sandbox pipeline functional
M4: Full Platform	16	All components integrated
M5: Testing Complete	18	All requirements validated
M6: Final Delivery	20	Thesis, code, presentation ready

1.7.6 Legal and Regulatory Feasibility

1.7.6.1 Licensing Considerations

Table 2.7.6.1: Licensing Considerations and Compliance

Component	License	Compliance Required
Cerberus Code	Apache 2.0	Open-source, attribution required
Sandbox	GPLv3	Source distribution if modified
Python Libraries	Various (MIT, BSD)	Attribution, no major restrictions
JavaScript Libraries	MIT	Minimal restrictions

Analysis: All licenses are compatible with academic/open-source distribution.

1.7.6.2 Privacy and Compliance

Data Handling:

- ☐ **No PII Collection:** System monitors file access patterns, not content
- ☐ **Local Processing:** All analysis occurs within controlled environment
 - ☐ **Ethical Testing:** Malware samples from approved academic sources only

Regulatory Considerations:

- ☐ **FERPA/GDPR Compliance:** No collection of personally identifiable information
- ☐ **Export Controls:** No cryptographic restrictions (basic TLS only)
- ☐ **Ethical Use:** Clear documentation for responsible deployment

1.7.6.3 Intellectual Property

- ☐ **University Policy:** Follows academic IP guidelines
- ☐ **Open Source:** Encourages community contribution and transparency
- ☐ **Publication Rights:** Maintained for academic papers and thesis

1.7.7 Risk Analysis and Mitigation

1.7.7.1 Identified Risks

Table 1.7.7.1: Identified Risks and Mitigation Strategies

Risk Category	Probability	Impact	Mitigation Strategy
Technical Complexity	Medium	High	Start with MVP, iterate
Performance Issues	Medium	Medium	Early benchmarking, optimization
Integration Failures	Low	High	Mock interfaces during development
Schedule Overruns	Medium	High	20% buffer time, clear milestones
Testing Challenges	High	Medium	Automated testing, peer review
Documentation Burden	High	Low	Document-as-you-go approach

1.7.7.2 Risk Mitigation Matrix

High Priority Risks:

- Sandbox Integration Failure**
 - Mitigation: Early prototype, fallback to static analysis
 - Monitoring: Weekly integration tests
- Performance Requirements Not Met**
 - Mitigation: Performance testing from Week 8
 - Fallback: Adjust requirements Technically infeasible

3. Schedule Compression

- a. Mitigation: Weekly progress reviews with advisor
- b. Contingency: Feature prioritization (must-have vs nice-to-have)

1.7.8 Feasibility Conclusion

1.7.8.1 Overall Assessment

Table 2.7.8.1: Overall Feasibility Assessment

Feasibility Dimension	Rating	Justification
Technical	HIGH	Proven technologies, clear integration path
Operational	MEDIUM-HIGH	Requires careful performance tuning
Economic	HIGH	Minimal costs, maximum academic benefit
Schedule	HIGH	20-week timeline with buffer achievable
Legal	HIGH	All open-source, no regulatory barriers
Overall	HIGH	Well-suited for graduation project

1.7.8.2 Recommended Actions

1. **Immediate Start:** Begin literature review and environment setup
2. **Early Prototyping:** Build detection agent prototype in first 4 weeks
3. **Continuous Testing:** Implement automated testing from Week 6
4. **Advisor Engagement:** Weekly progress reviews
5. **Documentation:** Maintain comprehensive records throughout

1.7.8.3 Success Probability: 85%

Factors Supporting Success:

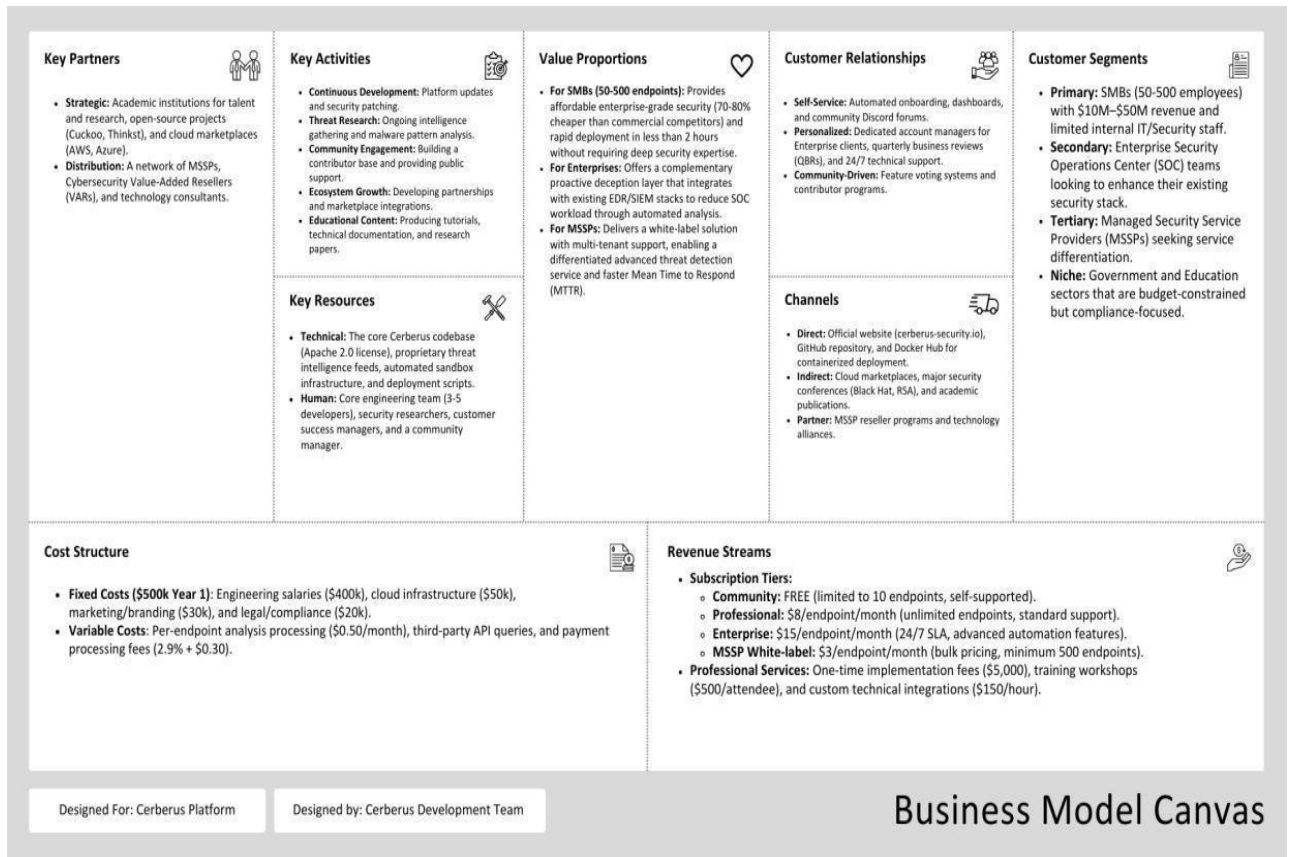
- ☐ Clear requirements and scope
- ☐ Available technology stack
- ☐ Academic support structure
- ☐ Modular, iterative approach
- ☐ Demonstrated proof-of-concept elements

Potential Challenges:

- ☐ Performance optimization for real-time monitoring
- ☐ Integration testing complexity
- ☐ Balancing features with timeline

2.7.8. Business Canvas

Figure 2.7.8: Business Model Canvas



1.8 Thesis Organization

Chapter 1: Introduction establishes the research context, motivation, and objectives. It articulates the problem statement Cerberus addresses, defines the complexity of the engineering challenge, enumerates constraints and standards governing development, and presents feasibility analysis validating the project's viability.

Chapter 2: Background provides foundational knowledge necessary to understand the Cerberus architecture. It surveys the evolution of cyber threats, limitations of traditional detection mechanisms, fundamentals of deception technology and behavioral malware analysis, and principles of automated incident response. This chapter ensures readers possess the domain knowledge to appreciate subsequent technical discussions.

Chapter 3: Related Work and Similar Systems positions Cerberus within the existing cybersecurity landscape. It reviews academic research in deception-based detection and automated malware analysis, examines commercial solutions (Thinkst Canary, Attivo Networks, CrowdStrike, Microsoft Defender), and identifies specific research gaps that Cerberus addresses. This comparative analysis demonstrates the novelty and contributions of the research.

Chapter 4: Analysis - Software Requirements Specification (IEEE 830) presents comprehensive requirements analysis following IEEE 830 standards. It includes system overview, use cases, system boundaries, functional requirements, non-functional requirements, threat modeling, risk assessment, and data flow diagrams. This chapter establishes the complete specification against which the implementation is evaluated.

Chapter 5: Design documents the architectural decisions and design alternatives considered during development. It presents UML diagrams including use case, sequence, class, component, and deployment diagrams; justifies technology stack selections; and describes the prototype implementation. Design rationale and trade-off analyses are explicitly documented.

Chapter 6: Implementation (not included in this SRS, documented separately) describes the actual construction of the Cerberus platform, implementation challenges encountered, solutions adopted, and lessons learned during development.

Chapter 7: Testing (not included in this SRS, documented separately) presents the test plan following IEEE 829 standards, including unit tests, integration tests, system tests, performance benchmarks, security assessments, and user acceptance testing results.

Chapter 8: Conclusion and Future Work (not included in this SRS, documented separately) synthesizes research findings, evaluates whether objectives were achieved, discusses limitations, and proposes directions for future enhancement and research.

Chapter 2 - Background

2.1 Introduction to Modern Cybersecurity Challenges

2.1.1 The Evolving Threat Landscape

The cybersecurity landscape has undergone a radical transformation in the past decade, evolving from simple viruses and worms to sophisticated, multi-vector attacks. According to the 2023 Verizon Data Breach Investigations Report, ransomware attacks increased by 13% year-over-year, surpassing all other threat categories in both frequency and financial impact. The average cost of a data breach reached \$4.45 million in 2023, representing a 15% increase over three years (IBM Security, 2023).

Modern cyber threats demonstrate several concerning characteristics:

- **Increased Sophistication:** Attackers employ advanced techniques including fileless malware, living-off-the-land binaries (LOLBins), and polymorphic code
- **Rapid Evolution:** New malware variants emerge daily, with antivirus vendors reporting over 450,000 new malicious programs daily (AV-TEST Institute, 2023)
- **Multi-Vector Approaches:** Modern attacks combine multiple techniques such as phishing, privilege escalation, lateral movement, and data exfiltration
- **Extended Dwell Time:** The median dwell time (time from compromise to detection) remains at 16 days for ransomware and 212 days for espionage attacks (FireEye, 2023)

2.1.2 Limitations of Traditional Security Approaches

Traditional signature-based detection systems, while effective against known threats, struggle against zero-day attacks and polymorphic malware. The reactive nature of conventional security operations—detecting threats after they have manifested—creates a fundamental disadvantage against proactive attackers. This detection gap has spurred interest in proactive defense mechanisms, including deception technology and behavioral analysis.

2.2 Deception Technology Foundations

2.2.1 Historical Context: Honeypots to Honeytokens

Deception technology traces its origins to the early 1990s with the development of honeypots—deliberately vulnerable systems designed to attract and study attackers. The seminal work of Spitzner (2003) in "Honeypots: Tracking Hackers" established foundational principles that remain relevant today. Early honeypots were primarily research tools, providing valuable insights into attacker behavior but requiring significant maintenance and posing operational risks.

The evolution from honeypots to honeytokens represented a significant paradigm shift. While honeypots are entire systems, honeytokens are discrete pieces of data—credentials, documents, or files—that serve as bait. This refinement reduced deployment complexity and expanded applicability across diverse environments.

2.2.2 Canary Files and Deceptive Documents

Canary files represent a specialized application of honeypot technology focused on file system monitoring. The concept, popularized by Thinkst Applied Research with their Canarytokens platform, involves placing specially crafted files in strategic locations where legitimate access is unlikely. These files contain embedded "tripwires" that trigger alerts when accessed.

Research by Bowen et al. (2009) demonstrated that deceptive documents could achieve detection rates exceeding 90% for insider threats while maintaining false positive rates below 5%. The psychological effectiveness of deception stems from attackers' natural tendency to seek valuable-seeming resources, making well-crafted decoys particularly effective.

2.2.3 Deception in Modern Security Operations

Contemporary deception technology has evolved beyond simple tripwires to encompass:

- **Network Deception:** Fake services, credentials, and shares to detect lateral movement
- **Endpoint Deception:** Canary files, registry keys, and configuration items
- **Data Deception:** Honeypots embedded within legitimate datasets
- **Active Defense:** Dynamic deception that adapts based on detected threats

The 2023 Gartner Market Guide for Deception Technology notes that organizations implementing comprehensive deception strategies reduce mean time to detection (MTTD) by 65% compared to traditional monitoring approaches.

2.3 Automated Malware Analysis

2.3.1 Static vs. Dynamic Analysis

Malware analysis techniques fall into two primary categories:

Static Analysis examines malware without executing it, using techniques such as:

- Signature matching (YARA rules)
- String extraction and entropy analysis
- Control flow graph reconstruction
- Packers and cryptors detection

Dynamic Analysis executes malware in controlled environments to observe behavior, including:

- System call monitoring
- Network traffic analysis
- File system and registry modifications
- Process creation and manipulation

Research by Egele et al. (2012) demonstrated that dynamic analysis captures approximately 85% of malicious behavior, while static analysis alone identifies only 40-60% of threats, highlighting the complementary nature of both approaches.

2.3.2 Sandbox Technology Evolution

The first generation of malware sandboxes emerged in the early 2000s, with systems like CWSandbox (2005) and Norman Sandbox (2006). These systems introduced automated

execution environments but suffered from limited evasion detection and scalability challenges.

Modern sandbox technology, exemplified by Sandbox (2010), Any.run (2015), and Joe Sandbox (2016), addresses these limitations through:

- **Multi-layered monitoring:** Combining API hooking, memory analysis, and network traffic inspection
- **Evasion resistance:** Detecting and countering anti-analysis techniques
- **Scalable architecture:** Supporting parallel analysis of multiple samples
- **Comprehensive reporting:** Generating detailed behavioral profiles and indicators of compromise (IOCs)

2.3.3 Sandbox: Architecture and Capabilities

Sandbox, selected for this project, represents an open-source standard in automated malware analysis. Its modular architecture consists of:

- **Host Machine:** Manages analysis tasks and virtual machine orchestration
- **Guest Virtual Machines:** Isolated environments for malware execution
- **Analysis Components:** Python modules for monitoring file system, registry, network, and process activities
- **Reporting Engine:** Generates structured JSON reports with behavioral signatures

Studies by Continella et al. (2016) found capable of analyzing over 10,000 samples monthly with detection accuracy exceeding 92% for common malware families. The platform's extensibility through custom signatures and analysis modules makes it particularly suitable for research applications.

2.4 Incident Response and Automation

2.4.1 The Incident Response Lifecycle

The National Institute of Standards and Technology (NIST) Computer Security Incident Handling Guide (SP 800-61) defines four primary phases of incident response:

1. **Preparation:** Developing policies, procedures, and technical capabilities
2. **Detection and Analysis:** Identifying and validating security incidents
3. **Containment, Eradication, and Recovery:** Limiting damage and restoring operations
4. **Post-Incident Activity:** Learning from incidents to improve future response

Research indicates that organizations with formal incident response programs experience 58% lower breach costs than those without (IBM, 2023).

2.4.2 Security Orchestration, Automation, and Response (SOAR)

SOAR platforms emerged in the late 2010s to address the growing volume and complexity of security alerts. These systems integrate three core capabilities:

- **Threat Intelligence Management:** Aggregating and correlating threat data
- **Security Operations Automation:** Automating repetitive tasks and workflows
- **Incident Response:** Providing case management and collaboration tools

A 2023 SANS survey found that organizations implementing SOAR capabilities reduced mean time to respond (MTTR) by 67% and increased analyst productivity by 45%.

2.4.3 Automated Response Challenges

While automation offers significant benefits, it introduces several challenges:

- **False Positive Risk:** Automated actions based on incorrect detections can disrupt legitimate operations
- **Attacker Adaptation:** Sophisticated attackers may trigger automated responses to create denial-of-service conditions
- **Ethical Considerations:** Automated countermeasures must comply with legal and regulatory frameworks
- **Complexity Management:** Automated workflows require careful design and ongoing maintenance

Research by Cavelti (2020) emphasizes the importance of human oversight in automated response systems, particularly for high-impact actions.

2.5 Integration Gaps in Current Security Solutions

2.5.1 Detection-Analysis-Response Disconnect

Despite advances in individual security technologies, significant gaps persist between detection, analysis, and response capabilities. Current security operations typically involve:

1. **Separate Tools:** Different systems for endpoint detection, network monitoring, and malware analysis
2. **Manual Processes:** Security analysts manually transferring data between systems
3. **Delayed Response:** Analysis results arriving hours or days after initial detection
4. **Context Loss:** Limited correlation between detection events and analysis findings

A 2023 Enterprise Strategy Group survey found that 68% of security operations centers (SOCs) reported difficulty correlating data from different security tools, with 42% citing integration challenges as their primary operational obstacle.

2.5.2 Academic Research on Integrated Security Systems

Academic literature reveals several attempts to bridge these gaps:

Dependence on Existing Platforms: Many research prototypes build on established platforms like CAPEv2 (formerly Cuckoo Sandbox) but focus on specific enhancements rather than comprehensive integration.

Limited Real-world Testing: Academic systems often demonstrate capabilities in controlled environments but lack validation in operational settings.

Focus on Specific Threat Types: Research frequently targets individual threat categories (e.g., ransomware) rather than developing generalized solutions.

2.5.3 Commercial Solution Limitations

Commercial security platforms, while feature-rich, present several limitations for academic research and practical deployment:

Cost Prohibitive: Enterprise deception and SOAR platforms often exceed \$100,000 annually, limiting accessibility for smaller organizations and research institutions.

Closed Architecture: Proprietary systems provide limited visibility into internal operations and restrict customization.

Integration Complexity: Commercial tools frequently require significant configuration and professional services for effective deployment.

Vendor Lock-in: Organizations become dependent on specific vendors for ongoing support and enhancements.

2.6 Theoretical Foundations

2.6.1 Game Theory in Cybersecurity

Deception technology finds theoretical grounding in game theory, particularly in asymmetric information games. The defender (organization) seeks to maximize detection while minimizing operational impact, while the attacker seeks to achieve objectives while avoiding detection. Research by Alpcan and Başar (2010) formalizes this interaction as a stochastic game, demonstrating that optimal defense strategies involve probabilistic deception placement.

2.6.2 Behavioral Analysis Principles

Automated malware analysis draws from several theoretical frameworks:

Control Flow Integrity: Ensuring program execution follows legitimate paths to detect code injection and manipulation.

Information Flow Tracking: Monitoring how data moves through systems to identify exfiltration attempts.

Anomaly Detection: Establishing baseline behavior and identifying deviations indicative of malicious activity.

Research by Wagner and Soto (2002) established foundational principles for behavioral malware detection, demonstrating that even simple system call sequences could identify malicious behavior with high accuracy.

2.6.3 System Design Principles

The integrated platform design incorporates several established software engineering principles:

Separation of Concerns: Distinct components handle detection, analysis, and response to facilitate testing and maintenance.

Loose Coupling: Modular architecture allows components to evolve independently.

Defense in Depth: Multiple detection mechanisms provide redundancy and comprehensive coverage.

Fail-Safe Design: Conservative automation defaults prevent catastrophic failures from false positives.

2.7 Related Academic Work

2.7.1 Integrated Deception Systems

Several academic projects have explored deception integration:

DECAF (Dynamic and Extensible Cyber Attack Framework) - University of California, Santa Barbara (2018): Combines honeypots with threat intelligence but lacks automated analysis capabilities.

ShadowNet - Carnegie Mellon University (2020): Implements network deception with machine learning-based adaptation but focuses exclusively on network-layer threats.

HoneyMix - University of Texas at Dallas (2021): Uses deceptive documents for insider threat detection but requires manual analysis of triggered events.

2.7.2 Automated Analysis Platforms

Academic contributions to malware analysis include:

VxClass - University of Bonn (2019): Machine learning-based malware classification system that achieves 96% accuracy but operates independently of detection systems.

Malware Analysis Workbench - Georgia Institute of Technology (2020): Integrates multiple analysis techniques but designed for manual researcher interaction rather than operational security.

Dynamic Analysis Pipeline - Purdue University (2021): Automates sample processing and reporting but lacks integration with endpoint detection systems.

2.7.3 Research Gap Identification

The literature review reveals several research gaps addressed by this project:

End-to-End Automation: Limited research on fully automated pipelines from detection through analysis to response.

Multi-Threat Coverage: Most systems specialize in specific threat types rather than providing comprehensive protection.

Practical Implementation: Few academic projects demonstrate operational viability beyond laboratory environments.

Open Architecture: Limited availability of integrated systems with transparent, customizable implementations.

Chapter 3 - Related Work and Similar Systems

3.1 Related Work

3.1.1 Academic Research in Deception Technology

“HoneyFiles: Deceptive Files for Intrusion Detection” (Yuill et al., 2004)

This foundational work introduced the concept of strategically placed decoy files designed to attract and detect attackers. The research demonstrated that honefiles trigger with high confidence when accessed, providing early warning of system compromise.

Contributions: Established theoretical foundations for file-based deception, demonstrating feasibility and effectiveness in detecting unauthorized access.

Limitations: Focused purely on detection without integration with analysis or response mechanisms. Manual investigation required upon triggers, limiting operational scalability.

Cerberus Extension: Builds upon honeyfile concepts but automates the complete workflow from detection through analysis to response, eliminating manual bottlenecks.

“Canarytokens: Large-Scale Deployment Strategies” (Thinkst Applied Research, 2015)

Thinkst Research pioneered canarytoken technology, creating various token types (DNS, HTTP, email) that trigger alerts when accessed from anywhere on the internet. The research validated large-scale deployment feasibility across enterprise environments.

Contributions: Demonstrated practical deployment of thousands of tokens with centralized alerting infrastructure. Proved extremely low false positive rates in production.

Limitations: Commercial implementation; limited academic publication of methodologies. Focused on alerting without deep threat characterization or automated response.

Cerberus Extension: Implements similar token concepts but open-source, with integration into behavioral analysis pipelines for comprehensive threat intelligence generation.

“UNVEIL: A Large-Scale Automated Approach to Detecting Ransomware” (Kharraz et al., 2016)

Research from Northeastern University developed behavioral detection specifically targeting ransomware through file system activity monitoring, detecting encryption patterns and file access anomalies.

Contributions: Demonstrated high detection accuracy (96.3%) for ransomware specifically through behavioral signatures. Published detailed methodology enabling replication.

Limitations: Specialized for ransomware only; does not detect spyware, credential theft, or lateral movement. Lacks proactive deception elements.

Cerberus Extension: Incorporates UNVEIL-inspired behavioral detection but extends coverage to multiple threat types (data stealers, lateral movement, insider threats) and combines with deception for earlier detection.

“CanaryTrap: Detecting Intruders by Creating Deceptive Network Environments” (Cohen et al., 2018)

Research exploring network-level deception by creating fake services, shares, and credentials visible to attackers conducting network reconnaissance.

Contributions: Demonstrated effectiveness of network deception in detecting lateral

movement and credential abuse attacks.

Limitations: Network-focused; limited endpoint deception capabilities. No integration with malware analysis sandboxes.

Cerberus Extension: Combines network deception concepts (decoy SSH keys, network shares) with endpoint-level canary files and automated sandbox analysis for multi-layered detection.

“ShapeShifter: Robust Physical Adversarial Attack on Faster R-CNN Object Detector” (Chen et al., 2018)

While focused on adversarial machine learning, this research highlighted how attackers manipulate systems to evade detection, informing defensive deception strategies.

Contributions: Detailed analysis of evasion techniques and adversarial thinking.

Limitations: Theoretical focus on ML systems rather than practical enterprise security.

Cerberus Application: Informs anti-evasion design in sandbox environments and deception asset placement strategies.

3.1.2 Academic Research in Malware Sandboxing

“CAPEv2 (formerly Cuckoo Sandbox) Sandbox: Automated Malware Analysis” (Guarnieri et al., 2012)

The original CAPEv2 (Configuration and Payload Extraction v2) paper detailed architecture for automated dynamic malware analysis with flexible extensibility.

Contributions: Open-source, widely adopted framework enabling practical malware research. Extensive monitoring capabilities and reporting infrastructure.

Limitations: Requires manual sample submission; no integration with detection systems.

Susceptible to sandbox evasion techniques without custom configurations.

Cerberus Extension: Automates integration with detection pipeline, submitting samples immediately upon canary triggers. Implements anti-evasion configurations documented in Chapter 5.

“Detecting Malware with Time Travel” (Kolbitsch et al., 2010)

Research introducing the concept of recording and replaying malware execution for analysis, addressing timing-based evasion techniques.

Contributions: Demonstrated effectiveness of deterministic replay for analyzing time-sensitive malware behaviors.

Limitations: Complex implementation; performance overhead limits practical deployment.

Cerberus Application: Informs design of timeout extensions and clock manipulation for patient malware detection.

“BareDroid: Large-Scale Analysis of Android Apps on Real Devices” (Bläsing et al., 2010)

While focused on mobile malware, this research demonstrated value of analyzing malware on bare-metal or native environments rather than hypervisors to avoid detection.

Contributions: Highlighted importance of realistic execution environments for comprehensive analysis.

Limitations: Bare-metal analysis sacrifices safety and scalability.

Cerberus Balance: Uses nested virtualization and artifact concealment to provide realistic environments while maintaining safety and isolation.

3.1.3 Academic Research in Automated Response

“SOTER: Automated IoT Safety and Security Analysis” (Celik et al., 2018)

Research developing automated methods for identifying and responding to IoT security vulnerabilities through static and dynamic analysis.

Contributions: Demonstrated feasibility of automated security policy enforcement in resource-constrained environments.

Limitations: IoT-specific; limited applicability to enterprise endpoint security.

Cerberus Parallel: Shares automation philosophy, adapting policy-based response to endpoint threat scenarios.

“Automated Incident Response Using AI” (Various Industry Whitepapers, 2019-2023)

Multiple industry research publications (IBM, Palo Alto Networks, CrowdStrike) have documented automated response implementations using machine learning.

Contributions: Demonstrated feasibility and ROI of automation in enterprise environments.

Limitations: Primarily proprietary implementations with limited technical disclosure. Heavy reliance on ML may introduce opacity in decision-making.

Cerberus Differentiation: Uses rule-based automation with transparent logic (if canary triggered AND sandbox confirms malicious, then contain) rather than opaque ML models, ensuring auditability.

3.2 Similar Systems

3.2.1. Commercial Deception Platforms

- **Thinkst Canary**

- **Description:** Cloud-managed deception platform deploying canary files, tokens, and honeypot devices across enterprise networks.
- **Key Features:** - Centralized cloud console for managing thousands of canaries - Pre-built canary templates (PDF, Office documents, scripts) - Extensive token types (DNS, HTTP, email, AWS keys, MySQL credentials) - Low false positive rate validated across thousands of deployments - Integration APIs for SIEM platforms
- **Advantages:** - Mature, production-ready platform with years of field validation - Minimal deployment and maintenance overhead - Excellent documentation and customer support
- **Limitations:** - **No Integrated Analysis:** Alerts require manual investigation to validate threats and understand attacker TTPs - **Separate Workflow:** Investigation happens outside platform; analysts must manually submit samples to sandboxes - **Cost:** Subscription pricing model (\$3,000-\$30,000+ annually depending on scale) - **Limited Behavioral Context:** Provides WHO accessed WHAT and WHEN but not detailed forensic timelines
- **Cerberus Advantage:** Automates the complete workflow from canary trigger through sandbox analysis to IOC extraction, providing comprehensive threat intelligence without manual investigation delays. Open-source nature eliminates licensing costs.

- **Attivo Networks (Now SentinelOne Singularity Identity)**

- **Description:** Enterprise deception platform with extensive decoy coverage (endpoints, servers, network, cloud) combined with lateral movement detection.
- **Key Features:** - Comprehensive deception portfolio: decoy workstations, servers, databases, cloud resources - Active Directory deception and credential analytics - Automated attacker engagement and forensic collection - Integration with endpoint protection platforms
- **Advantages:** - Enterprise-grade scalability (thousands of decoys) - Sophisticated deception strategies (machine learning-driven placement) - Deep integration with identity and access management systems
- **Limitations:** - **Complexity:** Deployment and tuning require specialized expertise and extensive planning - **Cost:** Enterprise pricing (\$50,000-\$500,000+ annually), prohibitive for SMBs or academic use - **Analysis Separation:** While collecting forensic data, malware analysis often requires external tools - **Heavyweight Infrastructure:** Significant server resources required for decoy hosting
- **Cerberus Advantage:** Lightweight, focused implementation suitable for SMBs and academic environments. Integrated sandbox analysis provides immediate threat characterization without complex infrastructure.

4.2.2. Commercial Malware Analysis Platforms

- **Any.Run**

- **Description:** Interactive online malware sandbox providing real-time, browser-based analysis with human-in-the-loop interaction capabilities.
- **Key Features:** - Interactive control over sandbox (click buttons, open files) - Real-time process and network monitoring - Extensive output formats (JSON, IOC lists, PCAP files) - Public and private analysis options - MITRE ATT&CK mapping
- **Advantages:** - Highly interactive, enabling dynamic investigation - Excellent UI/UX for analysts - Fast analysis startup (seconds) - Community repository of analyses
- **Limitations:** - **Manual Submission Required:** No automated integration with detection systems - **Cost:** Free tier limited; professional use requires subscription (\$3,000-\$15,000/year) - **Public Exposure Risk:** Free tier analyses are publicly visible, potentially exposing organizational samples - **No Response Integration:** Analysis endpoints at reporting; no automated containment
- **Cerberus Advantage:** Fully automated submission and analysis workflow. Self-hosted architecture ensures sample confidentiality. Integrated response executes containment without external intervention.
- **Hybrid Analysis (CrowdStrike)**
 - **Description:** Cloud-based malware analysis service leveraging CrowdStrike Falcon sandbox infrastructure.
 - **Key Features:** - Multi-environment analysis (Windows, Android, Linux) - Large-scale malware database for comparison - Detailed behavioral reports and IOC extraction - Integration with Falcon EDR platform
 - **Advantages:** - Deep integration with CrowdStrike ecosystem - Extensive threat intelligence correlation - Scalable cloud infrastructure
 - **Limitations:** - **Separate Platform:** Requires separate subscription from core Falcon EDR - **Manual Workflow:** Analysts must submit samples; not automatically triggered by detection events - **Cloud Dependency:** Requires internet connectivity and cloud service availability - **Proprietary:** Closed-source; limited customization
 - **Cerberus Advantage:** Tight integration between detection (canary triggers) and analysis () within single platform. Self-hosted deployment provides control and customization. Open-source foundation enables academic research and modification.

3.2.3. Commercial EDR Platforms

- **CrowdStrike Falcon**

- **Description:** Leading cloud-delivered endpoint protection platform with EDR, antivirus, and threat intelligence capabilities.
- **Key Features:** - Real-time behavioral detection across millions of endpoints - Machine learning-based prevention - Automated response and remediation (device isolation, process kill) - Extensive threat intelligence integration - Managed threat hunting services
- **Advantages:** - Industry-leading detection rates and low false positives - Minimal endpoint performance impact - Comprehensive telemetry and forensic capabilities - Professional incident response team support
- **Limitations:** - **No Native Deception:** Lacks built-in canary/honeypot capabilities -

Reactive Detection: Primarily detects threats after execution begins - **Analysis Separation:** Malware analysis requires separate Hybrid Analysis subscription - **Cost:** Enterprise pricing (\$8-\$15 per endpoint per month), cumulative costs significant for large deployments

- **Cerberus Differentiation:** Proactive deception-based detection complements reactive EDR monitoring. While CrowdStrike excels at post-execution detection, Cerberus triggers during reconnaissance and lateral movement phases.

- **Microsoft Defender for Endpoint**

- **Description:** Microsoft's enterprise endpoint protection platform integrated with Windows security infrastructure.
- **Key Features:** - Native Windows integration with minimal deployment overhead - Behavioral detection, antivirus, and vulnerability management - Automated investigation and remediation (AIR) - Integration with Microsoft 365 ecosystem - Threat analytics and attack surface reduction
- **Advantages:** - Included with certain Microsoft 365 licenses (cost-effective for existing customers) - Deep OS integration provides comprehensive visibility - Extensive automation capabilities - Regular updates aligned with Windows development
- **Limitations:** - **No Deception Capabilities:** Lacks honeyfile or canary token functionality - **Windows-Centric:** While Linux support exists, feature parity lags Windows - **Analysis Limited:** Basic automated analysis; complex malware requires manual expert review - **Lock-In:** Deep integration creates dependency on Microsoft ecosystem
- **Cerberus Differentiation:** Cross-platform support (Windows and Linux) with platform-agnostic deception. Open architecture enables integration with Defender while adding deception layer.

3.2.4. Summary Comparison

Table 3.2.4: Commercial Solutions Benchmark Comparison

Feature	Thinkst Canary	Attivo Networks	CrowdStrike Falcon	Cerberus
Deception Technology	Extensive	Comprehensive	None	Canary Files & Tokens
Automated Malware Analysis	Manual Required	Limited	Separate Platform	Integrated
Endpoint Monitoring	Alert Only	Forensic Collection	Full EDR	Process/File/Netw ork
Automated Response	None	Basic Containment	Advanced	Containment & Evidence
Cost	\$3K- \$30K/year	\$50K-\$500K/year	\$8- \$15/endpoint /month	Open-Source (Self- Hosted)
Deployment Complexity	Low	High	Medium	Medium
Cross-Platform	Partial	Yes	Windows- Centric	Windows & Linux
Customization	Limited	Moderate	Limited	Full (Open-Source)
MITRE ATT&CK Mapping	Manual	Yes	Yes	Automated
Threat Intelligence Export	API	STIX/TAXII	API	STIX, CEF, API

3.3 Research Gaps and Cerberus Innovation

3.3.1. Identified Gaps in Current Solutions

3.3.1.1. Gap 1: Integration Between Detection and Analysis

Current State: Deception platforms (Thinkst, Attivo) detect unauthorized access but stop at alerting. Analysts must manually validate threats, submit samples to sandboxes (Any.Run, Hybrid Analysis), wait for results, and then extract IOCs—a process taking 30-120 minutes per incident.

Cerberus Innovation: Eliminates the integration gap by creating a seamless pipeline: 1. Canary file accessed → Immediate detection 2. Malicious process identified → Automatic memory/file collection 3. Sample submitted to → Behavioral analysis begins 4. Analysis completes → IOCs extracted and ATT&CK mapped 5. Threat validated → Automated containment executes

Measurable Impact: Reduces mean time to validate threat from 30-120 minutes to <15 minutes with zero manual steps.

3.3.1.2. Gap 2: Lack of Comprehensive Multi-Threat Coverage

Current State: Solutions specialize by threat type: - **Ransomware Detection:** UNVEIL, ransomware-specific EDR features - **Data Theft Prevention:** DLP platforms - **Insider Threats:** UEBA systems - **Lateral Movement:** Network deception platforms

Organizations must deploy and integrate multiple specialized tools, each with separate consoles, alerting mechanisms, and response workflows.

Cerberus Innovation: Unified platform detecting multiple threat categories through strategic canary placement: - **Ransomware:** Encryption of canary files triggers ransomware-specific analysis - **Data Stealers/Spyware:** Access to fake credentials or sensitive document canaries - **Lateral Movement:** Decoy SSH keys, network share access - **Insider Threats:** After-hours access to canary files in sensitive directories

Measurable Impact: Single platform provides detection coverage previously requiring 3-5 specialized tools, reducing complexity and cost.

3.3.1.3. Gap 3: Automation Without Context

Current State: Automated response systems (SOAR platforms, EDR auto-remediation) execute containment actions based on confidence scores or rule matches but often lack detailed forensic context explaining WHY a system was quarantined.

Cerberus Innovation: Every automated response is accompanied by: - Complete process tree showing malware ancestry - Network timeline mapping C2 communications - File system activity log documenting malicious operations - Sandbox behavioral report with technique analysis - ATT&CK technique mapping contextualizing the attack

Measurable Impact: Reduces false containment anxiety by providing comprehensive justification for every automated action, improving trust in automation.

3.3.1.4. Gap 4: Enterprise Complexity vs. SMB Needs

Current State: Enterprise solutions (Attivo Networks, advanced EDR) offer comprehensive capabilities but require specialized expertise, extensive infrastructure, and significant budgets. SMBs and academic institutions cannot afford enterprise-grade deception and analysis capabilities.

Cerberus Innovation: Designed for mid-scale deployments (50-200 endpoints) with: - Docker-based deployment simplifying installation - Modest resource requirements (single server for sandbox, lightweight agents) - Open-source licensing eliminating per-endpoint costs - Intuitive dashboard requiring minimal specialized training

Measurable Impact: Democratizes advanced threat detection, making deception-integrated analysis accessible to organizations with limited security budgets.

3.3.1.5. Gap 5: Academic Research vs. Operational Reality

Current State: Academic papers demonstrate concepts (honeyfiles, behavioral detection, automated response) but rarely provide production-ready implementations. Commercial products offer operational maturity but limited technical disclosure for research purposes.

Cerberus Innovation: Bridges academia and industry by: - Fully documented open-source implementation enabling replication and study - Published performance metrics and detection accuracy measurements - Detailed threat modeling and security analysis - Real-world deployment scenarios and lessons learned

Measurable Impact: Advances cybersecurity research by contributing validated, replicable implementations to academic community while producing operationally viable tool.

3.3.2. Cerberus Unique Value Proposition

For Security Operations: - **Proactive Detection:** Identify threats during reconnaissance before damage occurs - **Automated Intelligence:** Transform detection into actionable threat intelligence without analyst effort - **Reduced Alert Fatigue:** High-confidence canary triggers replace low-confidence heuristic alerts - **Faster Response:** Automated containment reduces attacker dwell time

For Academic Research: - **Novel Architecture:** First documented integration of deception, sandboxing, and automated response - **Open Implementation:** Complete source code availability for study and extension - **Performance Benchmarks:** Rigorous testing and measurement enabling comparative research - **Methodological Contribution:** Framework for integrating disparate security technologies

For Small-Medium Organizations: - **Cost-Effective:** Open-source foundation eliminates licensing fees - **Simplified Deployment:** Docker containers and scripted installation reduce complexity - **Scalable Architecture:** Start small (10-50 endpoints), grow as needed - **Operational Value:** Provides enterprise-grade capabilities within SMB budgets.

This positioning establishes Cerberus as both an academic contribution advancing cybersecurity research and a practical tool addressing operational challenges faced by resource-constrained organizations. The following chapters detail how these innovations are realized through requirements analysis, architectural design, and prototype.

Chapter 4 - Analysis

4.1 Functional Requirements

4.1.1 Detection Module

4.1.1.1 Multi-Threat Detection

- The system **shall** detect ransomware activity by identifying specific encryption patterns executed on canary files.
- The system **shall** detect data exfiltration attempts by identifying unauthorized access to decoy documents (e.g., .docx, .pdf).
- The system **shall** detect credential theft attempts by monitoring access to decoy password files and fake SSH keys.
- The system **should** identify lateral movement attempts by detecting unauthorized access to network decoys and fake shares.

4.1.1.2 Real-Time File Monitoring

- The system **shall** monitor designated canary files in real-time for read, write, modify, and delete operations.
- The system **shall** correlate every file access event with the originating process ID, user context, and timestamp.
- The system **should** preserve the integrity of the deception environment by automatically restoring or rotating canary files immediately upon unauthorized modification.

4.1.2 Prevention Module

4.1.2.1 Host & Network Containment

- The system **shall** immediately terminate or suspend any process identified as malicious upon detection of a canary trigger.
- The system **shall** automatically isolate compromised endpoints from the network using host-based firewall rules to prevent lateral movement.
- The system **shall** block network data exfiltration by blacklisting IP addresses or domains identified as malicious C2 servers.

4.1.2.2 Policy Enforcement & Remediation

- The system **shall** enforce security policies at the agent level, including file access restrictions and credential usage controls.
- The system **should** execute automated remediation scripts, such as revoking compromised credentials or restoring modified files, following a confirmed incident.

4.1.2.3 Adaptive Deception & Hardening

- The system **should** dynamically adjust the placement and naming of canary files based on gathered threat intelligence and observed attack patterns.
- The system **should** automatically rotate or rename decoy assets in response to detected probing or reconnaissance attempts to maintain deception efficacy.
-

4.1.3 Analysis & Response Module

4.1.3.1 Sandbox Integration & Analysis

- The system **shall** automatically submit suspicious files and process artifacts to the CAPEv2 sandbox for dynamic behavioral analysis.

- The system **shall** track the status of sandbox analysis jobs and ingest the completed behavioral reports into the central database.
- The system **shall** parse analysis reports to store specific Indicators of Compromise (IOCs) and Tactics, Techniques, and Procedures (TTPs).

Threat Intelligence Enrichment (VirusTotal Integration)

The system **shall** perform a two-phase VirusTotal v3 enrichment pipeline for every analyzed threat. Phase 1 shall execute a parallel SHA-256 file hash lookup against the VirusTotal Files API immediately upon alert receipt, concurrent with CAPEv2 sample submission, to provide rapid reputation intelligence. Phase 2 shall enrich up to ten extracted network IOCs (IP addresses, domains, URLs) per incident via per-type VirusTotal v3 API calls following CAPEv2 analysis completion. Results shall be cached in Redis (24-hour TTL for file hashes, 6-hour TTL for network IOCs) to prevent redundant API requests for recurring samples. Enrichment detection ratios shall be mapped to a confidence value persisted in the IOC record, enabling analysts to prioritize response actions based on community-verified threat reputation data.

4.1.3.2 Alerting & Notification

- The system **shall** display real-time alerts on the web dashboard, providing immediate visibility into incident data and severity.
- The system **shall** maintain a searchable and persistent history of all alerts, filterable by date, affected host, and threat type.

4.1.3.3 Automated Forensic Collection

- The system **shall** automatically capture memory dumps of suspicious processes immediately upon threat detection.
- The system **shall** collect comprehensive forensic artifacts, including process trees, loaded modules, and active network connections.
- The system **shall** archive relevant file samples, registry changes, and system logs associated with each security incident for evidence preservation.

4.1.3.4 Threat Intelligence & Mitigation

- The system **shall** extract IOCs (IP addresses, domains, file hashes) from analysis reports and map identified threats to MITRE ATT&CK techniques.
- The system **should** generate actionable mitigation steps based on the severity and classification of the analyzed threat.
- The system **should** integrate with external Threat Intelligence (TI) feeds to enrich internal findings with contextual data.

4.1.4 Management & Administration Module

4.1.4.1 Agent Management

- The system **shall** secure agent registration and enforce mutual authentication between the endpoint agents and the central server.
- The system **shall** continuously monitor the health, version status, and connectivity of all deployed endpoint agents.
- The system **should** enable the push of remote configuration updates and security policy changes to agents without requiring re-installation.

4.1.4.2 Canary File Management

- The system **shall** automatically deploy canary files to specified directories on monitored endpoints.
- The system **shall** support the generation of multiple decoy file types (e.g., Office documents, PDFs, text files) using customizable content templates.
- The system **should** provide API endpoints to allow for dynamic querying of canary status and on-demand deployment.

4.1.4.3 Dashboard & Reporting

- The system **shall** provide an intuitive web interface for the visualization of real-time threats, including statistical charts and incident timelines.
- The system **shall** present detailed sandbox analysis results, offering drill-down views into behavioral logs and process activities.
- The system **shall** generate comprehensive incident reports that compile attack timelines, forensic evidence, and automated response actions.

4.1.4.4 Incident Workflow Management

- The system **shall** track the complete lifecycle of an incident from detection through investigation to containment and resolution.
- The system **should** support analyst collaboration by enabling case assignment and the revision of incident tags and categories.

4.1.4.5 Audit & Compliance

- The system **shall** maintain tamper-proof audit logs of all detection events, containment actions, and administrative modifications.
- The system **shall** track the chain of custody for all collected forensic evidence to ensure data integrity.
- The system **should** support data export in standard compliance formats (e.g., SIEM CEF, STIX/TAXII) for integration with external security systems.

4.2 Non-Functional Requirements

4.2.1 Performance Requirements

4.2.1.1 CPU Utilization

Requirement ID: NFR-PERF-001

Description: Endpoint agent CPU utilization **shall not** exceed 8% average during normal monitoring operations as measured by Windows Performance Monitor (perfmon.exe) or Linux top command over 1-hour observation windows. **[CRITICAL]**

Measurement Tool: Windows Performance Monitor, Linux top/htop

Measurement Method: Record CPU% every 60 seconds for 1 hour, calculate arithmetic mean

Acceptance Criteria: Mean $\leq 8.0\%$

Test Environment: Windows 10 with 4GB RAM, 2 CPU cores; Ubuntu 20.04 with 4GB RAM.

4.2.1.2 Detection Latency

Requirement ID: NFR-PERF-002

Description: Detection latency from canary file access to alert arrival at Central Server **shall not** exceed 10 seconds under normal network conditions ($\leq 20\text{ms}$ latency, $< 1\%$ packet loss). **[CRITICAL]**

Measurement Tool: Custom timing instrumentation, Prometheus metrics

Measurement Method: Timestamp at file access, timestamp at server receipt, calculate delta

Acceptance Criteria: p95 latency ≤ 10 seconds

Test Environment: LAN connection, 1 Gbps, $< 5\text{ms}$ latency

4.2.2 Security Requirements

4.2.2.1 Communication Encryption

Requirement ID: NFR-SEC-001

Description: All agent-to-server communications **shall** be encrypted using TLS 1.2 or higher with minimum 2048-bit RSA or 256-bit ECC certificates, as verified by Wireshark network traffic analysis and OpenSSL verification. **[CRITICAL]**

Measurement Tool: Wireshark packet capture, OpenSSL s_client

Measurement Method: Capture agent-server traffic, verify TLS handshake and encryption

Acceptance Criteria: 100% of connections use TLS 1.2+, no plaintext data observed

Certificate: Self-signed acceptable for prototype, CA-signed recommended for production

4.2.3 Reliability Requirements

4.2.3.1 Agent Auto-Recovery

Requirement ID: NFR-REL-001

Description: Endpoint agents **shall** automatically restart within 90 seconds of crash or termination, as implemented by OS service manager (Windows Service Control Manager, Linux systemd). **[HIGH]**

Measurement Tool: Service logs, watchdog monitoring

Measurement Method: Forcibly terminate agent, measure time to restart

Acceptance Criteria: Agent fully operational ≤ 90 seconds post-crash

Test Scenarios: 10 forced terminations, all must recover

4.2.4 Scalability Requirements

4.2.4.1 Endpoint Support Capacity

Requirement ID: NFR-SCAL-001

Description: The system **shall** support 50-200 concurrent monitored endpoints without degradation in detection latency (maintaining <10 second alert delivery) or agent performance (maintaining $<8\%$ CPU usage). **[CRITICAL]**

Measurement Tool: Load testing framework, performance monitoring

Measurement Method: Deploy agents to 50, 100, 150, 200 endpoints, measure performance

Acceptance Criteria: All performance NFRs met at 200 endpoints

Infrastructure: Central Server scaled appropriately (8+ cores, 16+ GB RAM)

Chapter 5 – Design

5.1. Design Alternatives

5.1.1 Iteration 1: Basic Canary File Monitor (Weeks 1-3)

The initial design prioritized simplicity to validate core detection concepts.

- **Initial Architecture:** A standalone Python script on the endpoint writing alerts to a local alerts.txt file.
- **Requirements Addressed:** Partial coverage of **FR-001** (Multi-threat detection) and **FR-002** (Real-time monitoring).
- **Identified Limitations:** Lack of central visibility for the SOC, no automated analysis, and a requirement for 100% manual response.

5.1.2 Iteration 2: Centralized Alerting System (Weeks 4-6)

This phase introduced centralized management to address the visibility gap.

- **Enhanced Architecture:** Endpoint agents communicating with a Flask Web Server backed by an SQLite database and a Web Dashboard.
- **Design Changes:** Added REST API endpoints, alert persistence, and encrypted transmission (**NFR-SEC-001**).
- **Identified Limitations:** Alerts lacked behavioral context; manual analysis still created significant response latency.

5.1.3 Iteration 3: Automated Analysis Integration (Weeks 7-10)

A major architectural pivot was made to integrate deep behavioral analysis.

- **Major Pivot:** Integration with **Sandbox** for automated sample submission and report processing.
- **Critical Design Decision:** Selected direct REST API integration over an abstract layer to reduce complexity and align with the project timeline.
- **Requirements Addressed:** **FR-004** (Sandbox integration), **FR-003** (Forensic collection), and **NFR-PERF-007** (Analysis throughput).

5.1.4 Iteration 4: Multi-Threat Detection Expansion (Weeks 11-12)

The platform evolved from ransomware-only detection to a multi-threat defense system.

- **Logic Evolution:** Implemented case-based detection for Ransomware, Data Stealers, Credential Theft, and Lateral Movement.
- **Design Changes:** Developed threat-specific logic and tailored response actions for each malware category.

5.1.5 Iteration 5: Automated Response Pipeline (Weeks 13-14)

The final development iteration focused on closing the loop between detection and containment.

- **Process Flow:** Evolved to a "Detect → Collect → Analyze → Respond" automated pipeline.
- **Critical Design Decision:** Chose a **Conservative Policy** (alert-first) to minimize false-positive disruption while allowing for an "Aggressive Mode" if validated.
- **Requirements Addressed:** **FR-007** (Automated containment) and **NFR-REL-002** (Agent auto-recovery).

5.2 Final Architecture (Week 15)

5.2.1 Integrated Platform Design

The final design consists of four integrated layers:

- **Agent Layer:** Handles monitoring, process tracking, and automated response.

- **Server Layer:** Manages alerts, forensic storage, and agent configurations.
- **Analysis Layer:** Extracts IOCs and maps threats to the **MITRE ATT&CK** framework.
- **Dashboard:** Provides real-time visualization and incident management.

5.3 Design Rationale and Trade-offs

5.3.1 Key Design Decisions

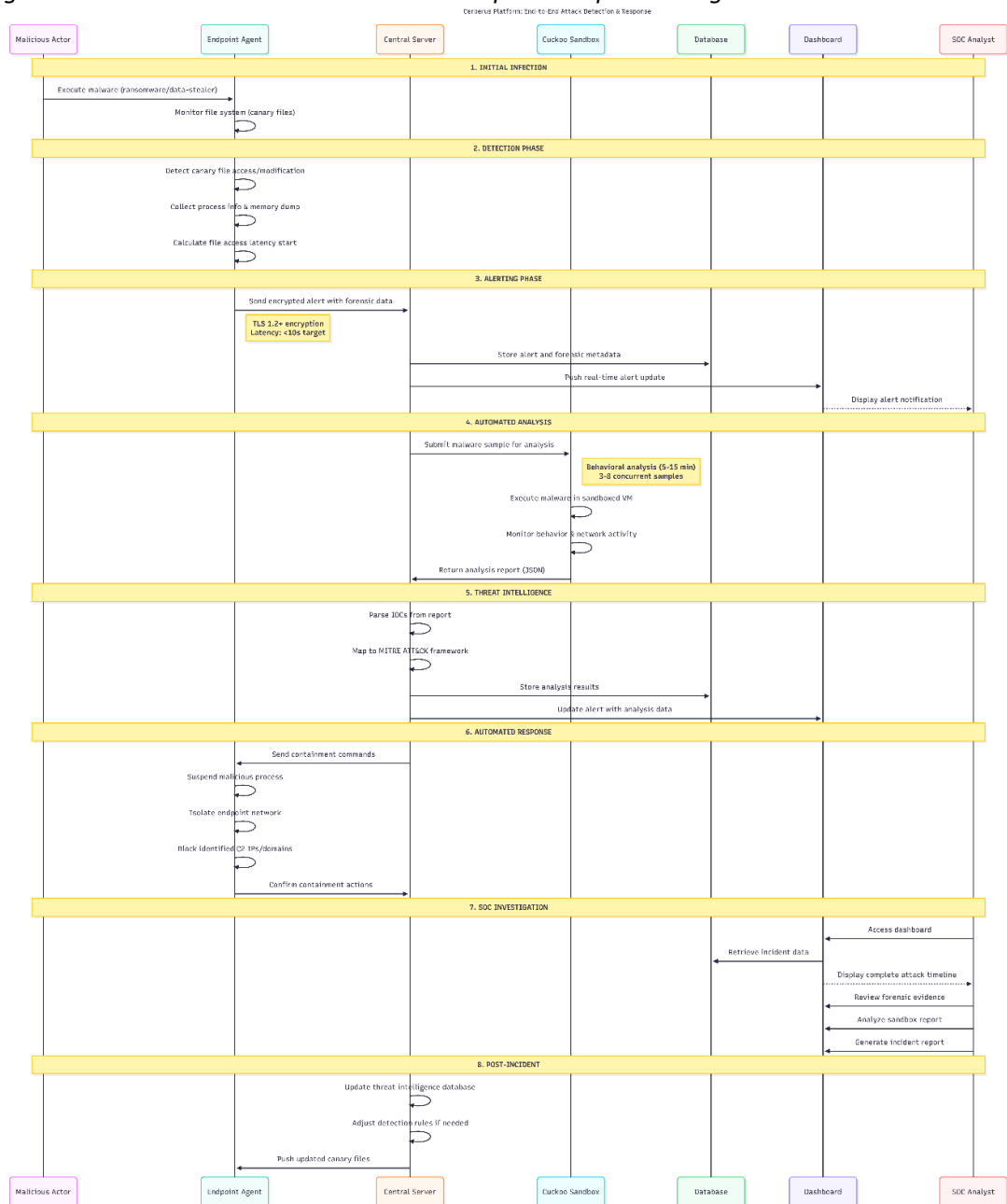
- **Agent Architecture:** Chose **User-mode service** over a Kernel-mode driver to reduce development risk, mitigated by a watchdog for auto-recovery.
- **Communication:** Selected **RESTful API** for firewall-friendly operations, supplemented by WebSockets for real-time dashboard updates.
- **Database:** Selected **PostgreSQL** because forensic and incident data are inherently relational.

5.3.2 Technical Debt Management

- **Deliberate Simplifications:** Utilized a migration path from SQLite to PostgreSQL and prioritized static canaries with manual rotation over complex ML-driven deployment.
- **Future Extensions:** The design allows for future vendor-neutral sandbox interfaces and distributed server architectures.

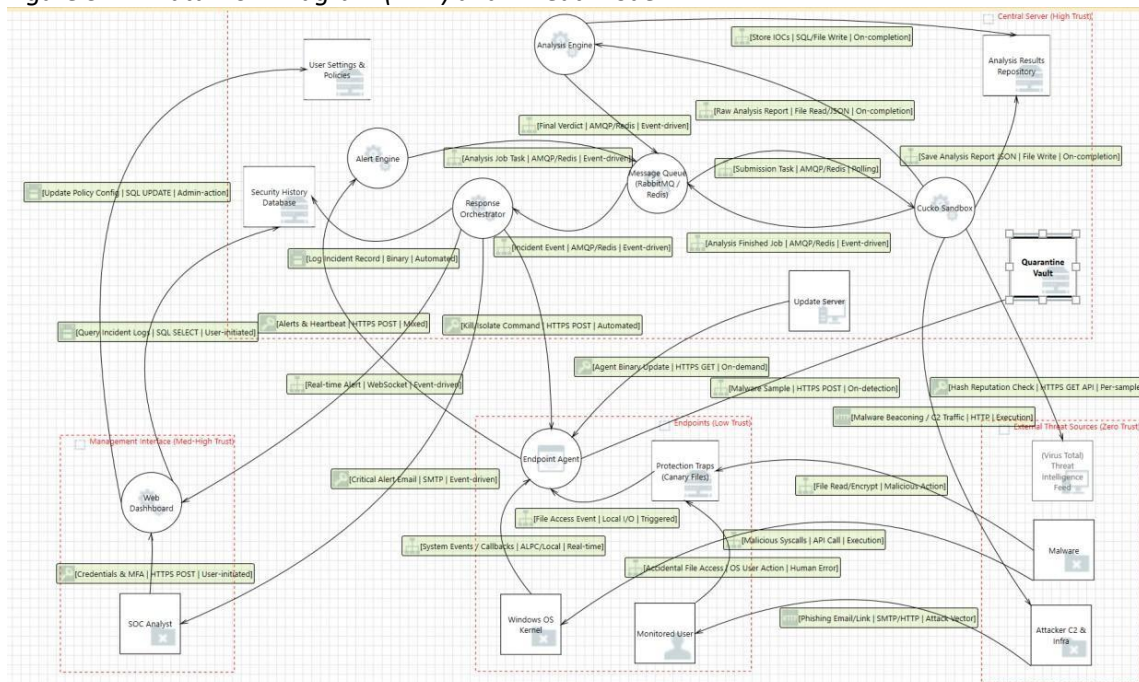
5.2.3. Sequence Diagram

Figure 5.2.3: End-to-End Attack Detection & Response Sequence Diagram



5.2.4. DFD(Threat Model)

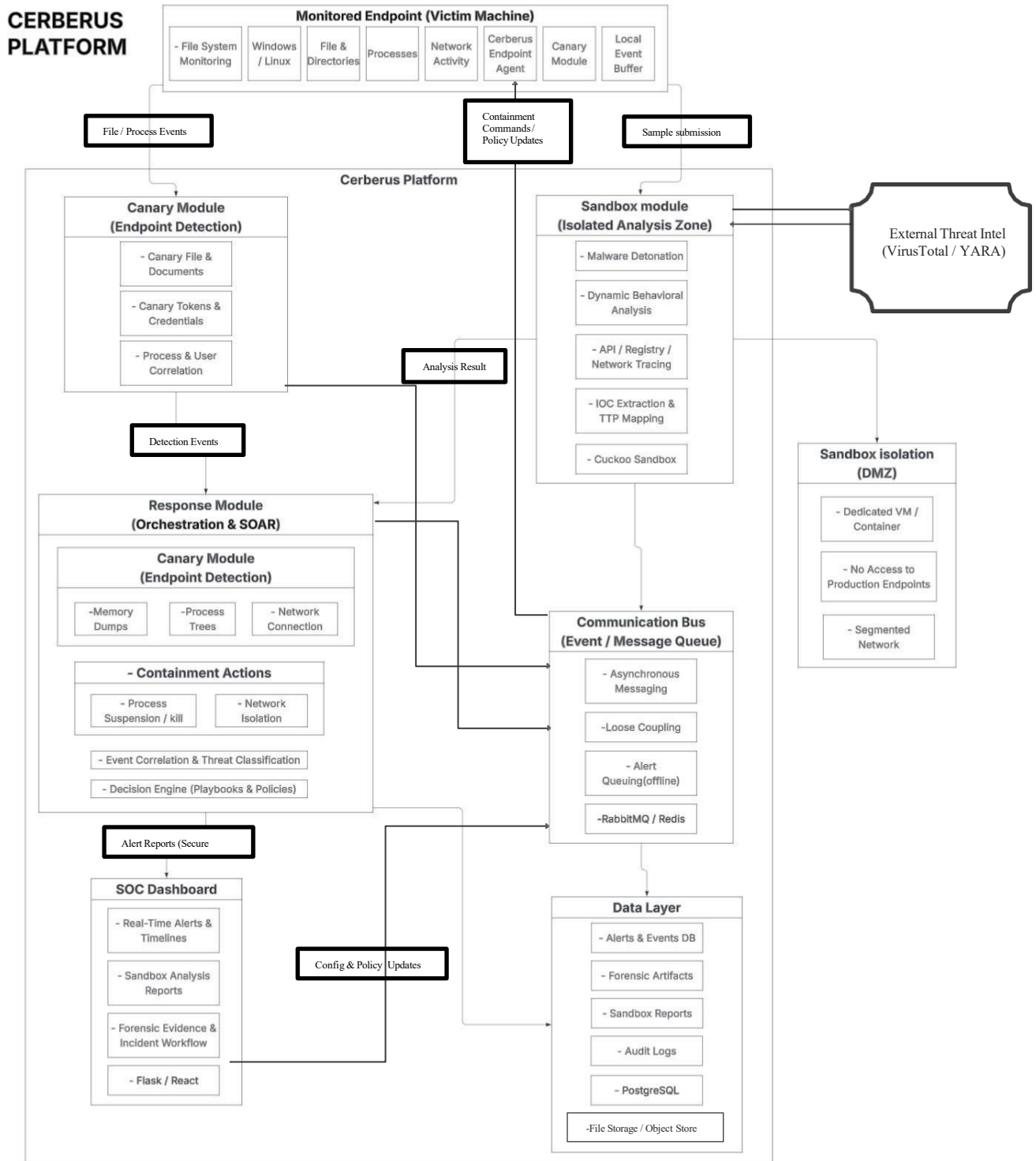
Figure 5.2.4: Data Flow Diagram (DFD) and Threat Model



	A	B	C	D	E	F	
	Threat Group	IDs	Category (STRIDE)	Component	Risk Score	Mitigation	Residual Risk
1	Response Command Injection	29, 32, 61	Spoofing / Tampering	Response Orchestrator	(10/10) CRITICAL	Command Signing: Use Python's hashlib to sign commands with a secret key. Agents verify the hash before execution.	(4/10) Low-Med
2	Sandbox Escape	6, 14	Elevation of Privilege	Cuckoo Sandbox	(10/10) CRITICAL	Network Isolation: Use Host-Only networking in VirtualBox to prevent malware from reaching the internet or local network.	(6/10) Med-High
3	Endpoint Agent Spoofing	19, 109	Spoofing	Endpoint Agent	(9/10) CRITICAL	Unique API Keys: Assign a static UUID to each agent. The server rejects any request without a valid key in the header.	(5/10) Medium
4	Message Queue Elevation	3, 7, 26	Elevation of Privilege / Information Disclosure	Message Queue	(9/10) CRITICAL	Internal Authentication: Set a strong password for Redis/RabbitMQ and bind it to 127.0.0.1 so only local services can talk to it.	(4/10) Medium
5	Dashboard Auth Bypass	63, 64, 127	Spoofing / Information Disclosure	Web Dashboard	(9/10) CRITICAL	Session Security: Use Flask's built-in session management with HttpOnly and Secure flags enabled.	(4/10) Low-Med
6	Database Compromise	72, 133	Tampering / Information Disclosure	Security Database	(9/10) CRITICAL	Encryption at Rest: Use the Python cryptography library (Fernet) to encrypt sensitive columns (like IPs) before saving.	(5/10) Medium
7	Analysis Results Tampering	1, 15, 97	Tampering	Analysis Results	(8/10) HIGH	File Permissions: Set the repository folder to "Append-Only" or Read-Only for the web user to prevent modification of old reports.	(3/10) Low
8	Insider Threat	92, 127	Elevation of Privilege / Tampering	Dashboard / Settings	(8/10) HIGH	Basic RBAC: Create simple logic in the dashboard to show/hide buttons based on the user's role (Admin vs. Analyst).	(6/10) Med-High
9	Canary Detection	85, 91, 131	Spoofing	Protection Traps	(7/10) HIGH	Realistic Naming: Use a simple script to name files based on real document names (e.g., invoice_2024.pdf) instead of "honey_file".	(6/10) Medium
10							

5.2.5. Design modular components (sandbox, canary, response)

Figure 5.2.5: Cerberus modular components



5.3. Technologies and Tools Used

Figure 5.3: Technologies and Tools Stack

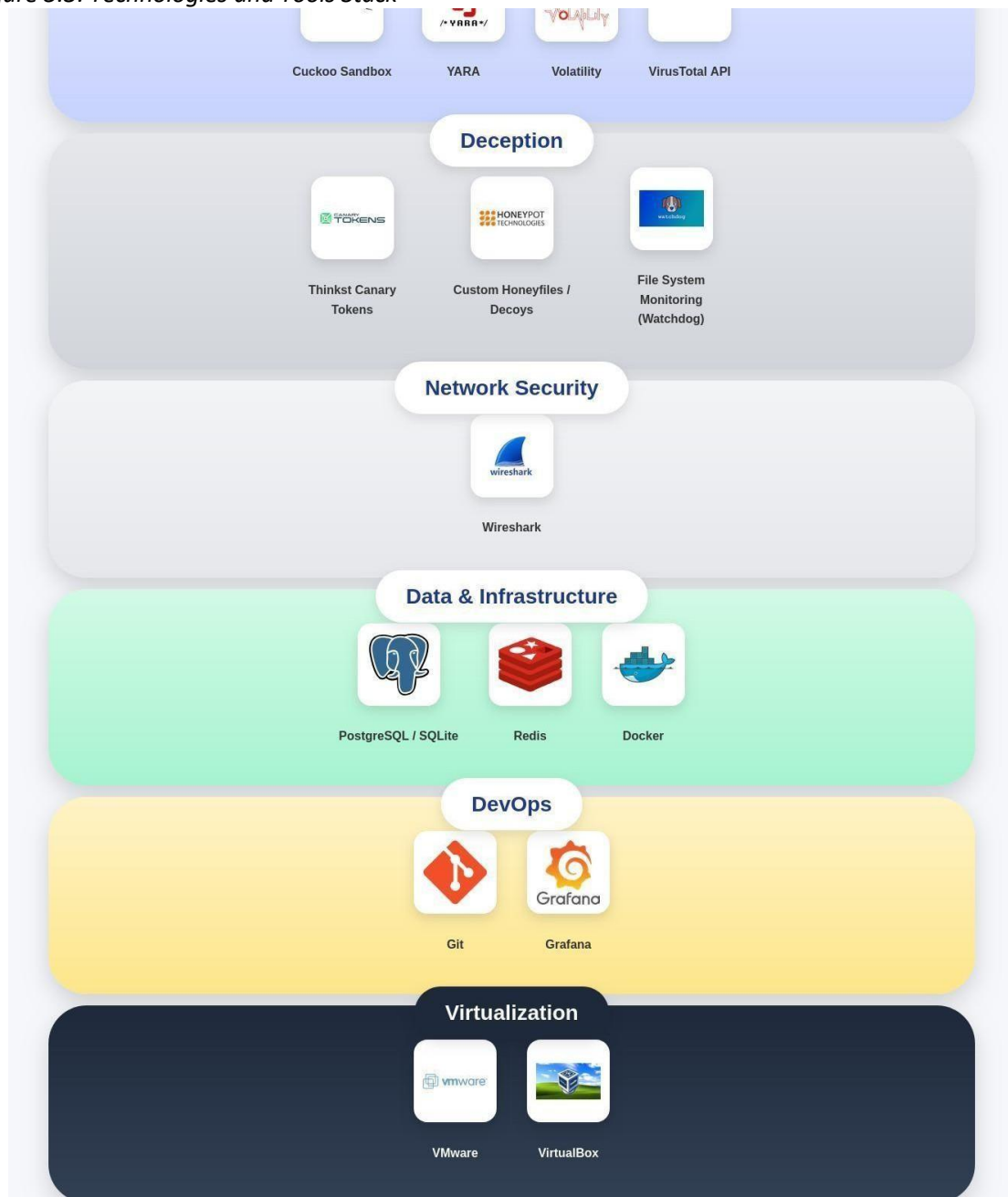


Table 5.3: Detailed Technology Stack and Libraries

Layer	Technology	Version	License	Purpose
Agent Language	Python	3.8-3.11	PSF	Cross-platform endpoint monitoring
Agent Monitoring	Watchdog	2.1+	Apache 2.0	File system events
Agent Forensics	psutil	5.9+	BSD	Process/memory inspection
Server Framework	Flask	2.3+	BSD	Web dashboard and API
Database	PostgreSQL	13+	PostgreSQL	Alert/incident storage
Message Queue	Redis	6.2+	BSD	Async job processing
Sandbox		2.0.7+	GPLv3	Malware analysis
Pattern Matching	YARA	4.2+	BSD	Signature detection
Memory Forensics	Volatility	2.6/3.0	GPLv2	Memory analysis
Threat Intel	VirusTotal API	v3	Proprietary	IOC enrichment
Network Analysis	Wireshark	3.6+	GPLv2	Packet capture
Containerization	Docker	20.10+	Apache 2.0	Deployment
Virtualization	VirtualBox	6.1+	GPLv2	Sandbox isolation
Visualization	Chart.js	3.9+	MIT	Dashboard charts
Frontend	Bootstrap	5.2+	MIT	Responsive UI
Version Control	Git	2.30+	GPLv2	Code management
Encryption	cryptography	3.4+	Apache/BSD	Evidence protection
HTTP Client	requests	2.28+	Apache 2.0	Agent-server comm
Deception	Thinkst Canary Tokens	N/A	Free Tier	Network deception
Document Gen	python-docx, openpyxl	Latest	MIT	Honeyfile creation

5.4. Prototype

5.4.1. SOC Dashboard

- Real-time threat intelligence with live incident tracking
- Key Performance Indicators showing active threats, response times, and protected endpoints
- Interactive threat timeline visualizing attack patterns over the last 24 hours
- Severity distribution charts categorizing threats by criticality level
- Active incidents table with color-coded severity indicators and containment status
- Network threat map showing attack origins and endpoint relationships

Figure 5.4.1: SOC Dashboard Overview



Figure 5.4.1: Active Incidents Interface

Active Incidents

ID	Time	Severity	Type	Endpoint	Status	Action
INC-1000	2025-12-23 03:10:28	Medium	Data Stealer	SRV-47	▲ Contained	Auto-contained
INC-1001	2025-12-24 23:10:28	Low	DDoS	WS-112	▲ Resolved	Needs review
INC-1002	2025-12-24 14:10:28	Low	Data Stealer	WS-111	▲ Contained	Escalated to L2
INC-1003	2025-12-22 19:10:28	Low	Malware Dropper	DB-04	▲ Investigating	Needs review
INC-1004	2025-12-24 14:10:28	High	Ransomware	WS-119	▲ Resolved	Needs review
INC-1005	2025-12-24 20:10:28	Medium	Lateral Movement	WS-111	▲ Resolved	Needs review
INC-1006	2025-12-23 02:10:28	Medium	DDoS	DB-05	▲ Detected	Needs review
INC-1007	2025-12-23 23:10:28	Critical	Data Stealer	WS-105	▲ Investigating	Auto-contained
INC-1008	2025-12-23 06:10:28	Low	Insider Threat	WS-104	▲ Resolved	Auto-contained
INC-1009	2025-12-23 18:10:28	Medium	Ransomware	DB-01	▲ Detected	Monitoring

5.4.2. Malware Sandbox Analysis

Advanced behavioral analysis workspace featuring:

- Live sandbox queue showing malware samples being analyzed with progress tracking
- Interactive process tree visualization mapping malware execution chains
- Behavioral timeline displaying API call sequences and system interactions
- Indicators of Compromise (IOC) extraction panel with MITRE ATT&CK mapping
- Risk scoring system with detailed behavioral analysis metrics

- **Sample submission interface** for manual analysis requests

Figure 5.4.2: Malware Sandbox Analysis Interface

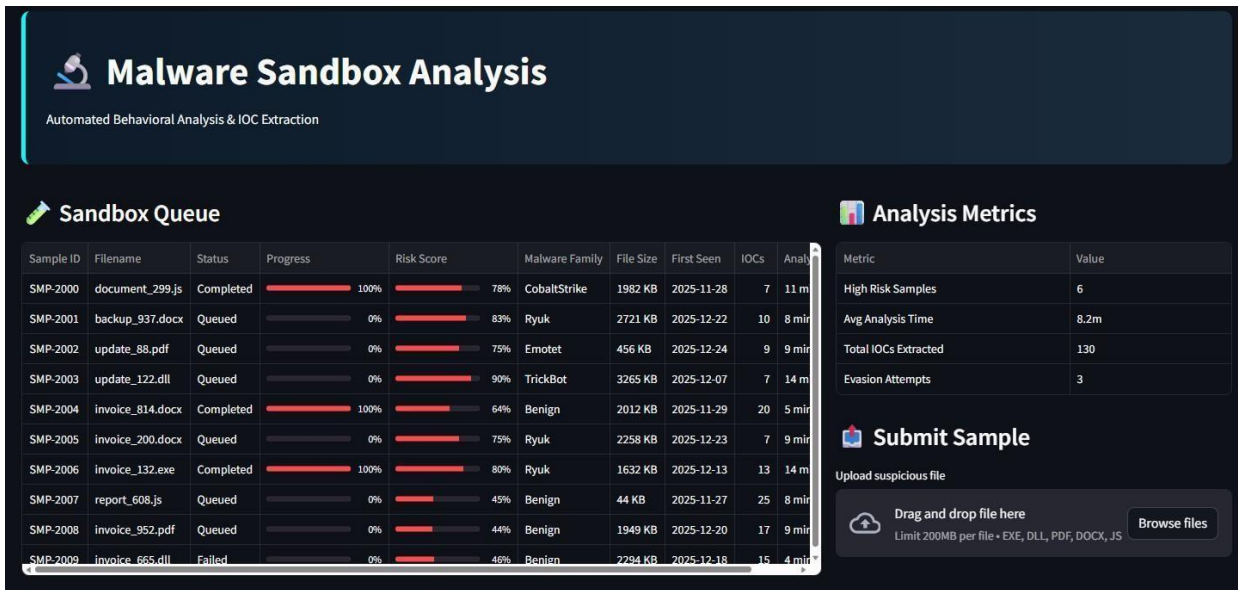
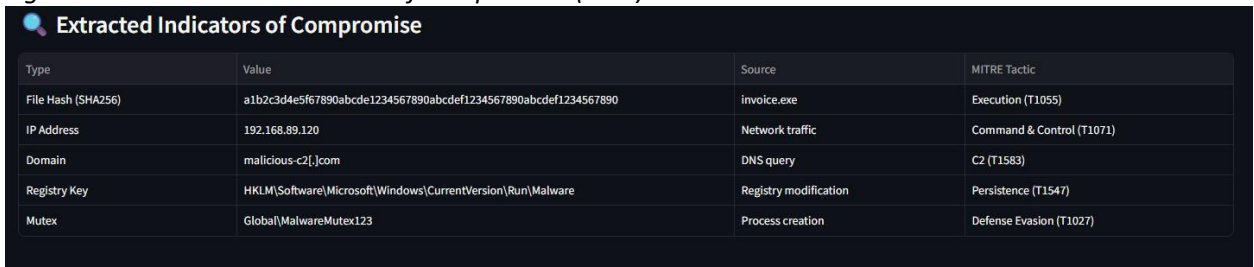


Figure 5.4.2: Extracted Indicators of Compromise (IOCs) Panel



5.4.3. Security Report Generator

Professional reporting module offering:

- **Configurable report templates** for incident summaries, forensic analysis, and compliance documentation
- **Time-range selectors** for historical data analysis
- **Live report preview** with automatic section population
- **Multiple export formats** including PDF, CSV, HTML, and threat intelligence standards (STIX/TAXII)
- **Automated evidence compilation** from incident data and analysis results

Figure 5.4.3: Security Report Generator

Security Report Generator

Automated Incident Reporting & Documentation

Report Configuration

Report Type: Incident Report

Time Range: Last 24 hours

Sections to Include:

- ☒ Executive Summary
- ☒ Timeline of Events
- ☒ Indicators of Compromise
- ☒ MITRE ATT&CK Mapping
- ☒ Recommendations
- ☐ Appendix (Raw Data)

Export Format:

- ☒ PDF (Professional)
- ☐ HTML (Interactive)
- ☐ CSV (Raw Data)
- ☐ STIX/TAXII (Threat Intel)

Generate Report

Report Preview

Configure report settings and click 'Generate Report' to preview

5.4.4. Deception Canary Management

Honeypot deployment and monitoring interface:

- **Canary file deployment controls** with customizable file types and locations
- **Real-time canary status dashboard** showing triggered vs. active deception assets
- **Geographic deployment map** visualizing canary distribution across network segments
- **Access pattern analytics** tracking attacker interactions with decoy resources
- **Automated canary rotation** and lifecycle management features

Figure 5.4.4: Deception Canary Management Dashboard

Deception Canary Management

Honeypot File Deployment & Monitoring

Total Canaries: 20

Triggered Today: 3

Coverage: 70%

Avg Response Time: 2.1s

Deploy New Canary

File Name: confidential_financial_report.xlsx

Deployment Location: /Documents/

Content Template: Financial Data

File Type: Office Document

Sensitivity Level: Low

Expiration Date: 2025/12/24

Deploy Canary

Canary Status Overview

ID	Filename	Type	Location	Deployed	Status	Last Accessed	Access Count
CAN-3000	backup_financial_report.xlsx	Office Document	/Documents/	2025-10-06	Triggered	2025-12-24 17:16	2%
CAN-3001	financial_ssh_keys.tar	Credentials	/Documents/	2025-09-29	Active	Never	0%
CAN-3002	financial_database_backup.sql	Database	/Documents/	2025-10-16	Active	Never	0%
CAN-3003	financial_ssh_keys.tar	Database	/Shared/	2025-12-24	Active	Never	0%
CAN-3004	financial_financial_report.xlsx	Archive	/AppData/	2025-10-08	Active	Never	0%
CAN-3005	secret_database_backup.sql	Credentials	/Shared/	2025-09-27	Triggered	2025-12-23 22:16	4%
CAN-3006	confidential_client_passwords.txt	Database	/AppData/	2025-09-26	Active	Never	0%

5.4.5. Agent Activity Monitoring

Endpoint protection agent management console:

- **Agent health dashboard** showing online/offline status and system resource usage
- **Real-time log viewer** with filtering by severity level and agent identity
- **Performance metrics** tracking CPU, memory, and log queue statistics
- **Configuration management** for remote agent updates and policy enforcement
- **Agent communication status** with last check-in timestamps and connectivity indicators

Figure 5.4.5: Agent Health and Activity Logs

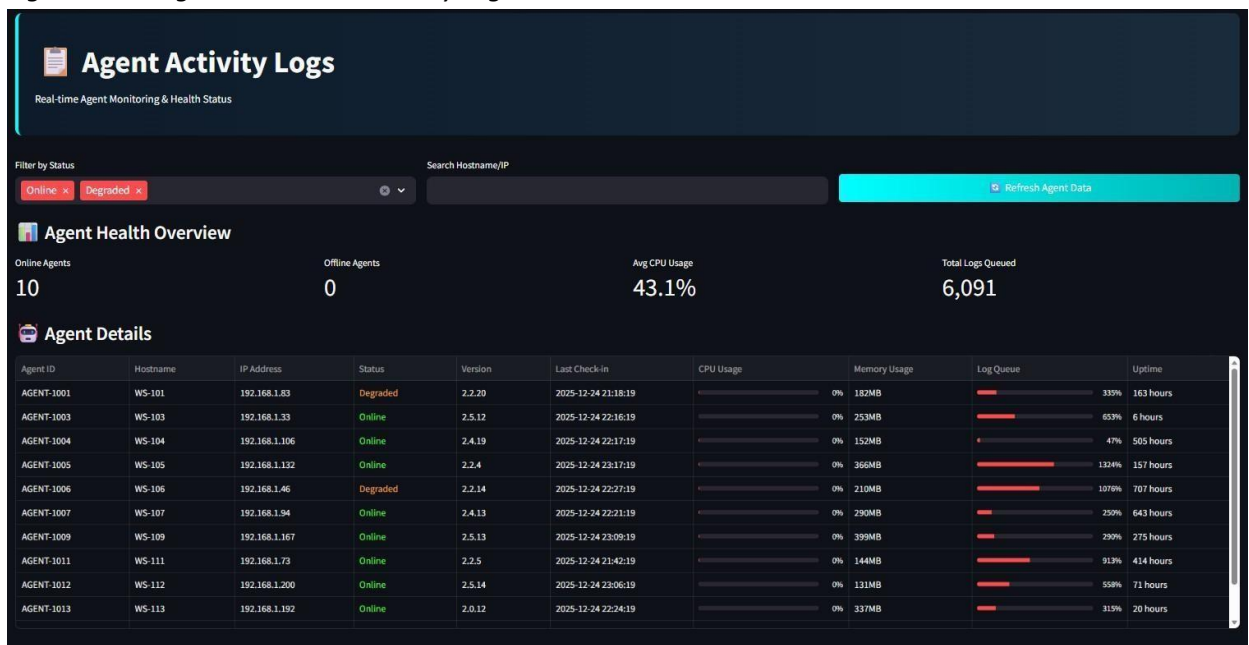
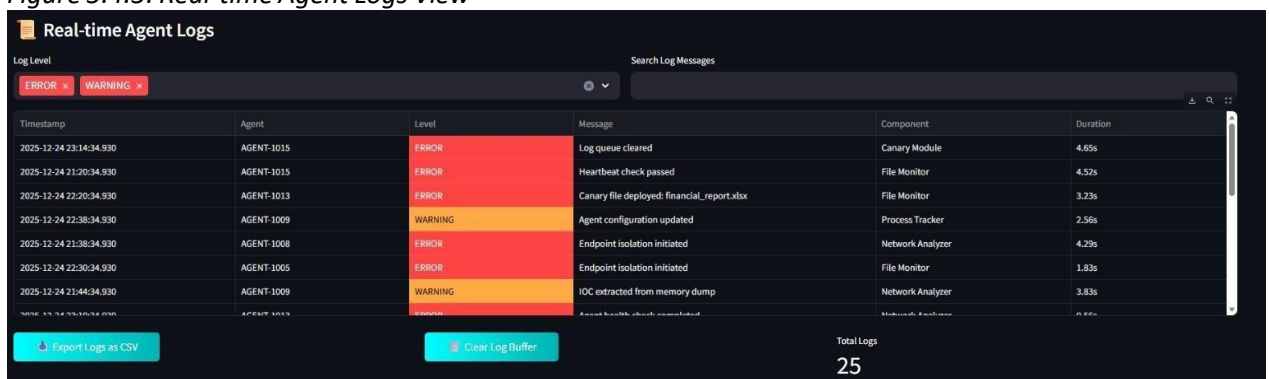


Figure 5.4.5: Real-time Agent Logs View



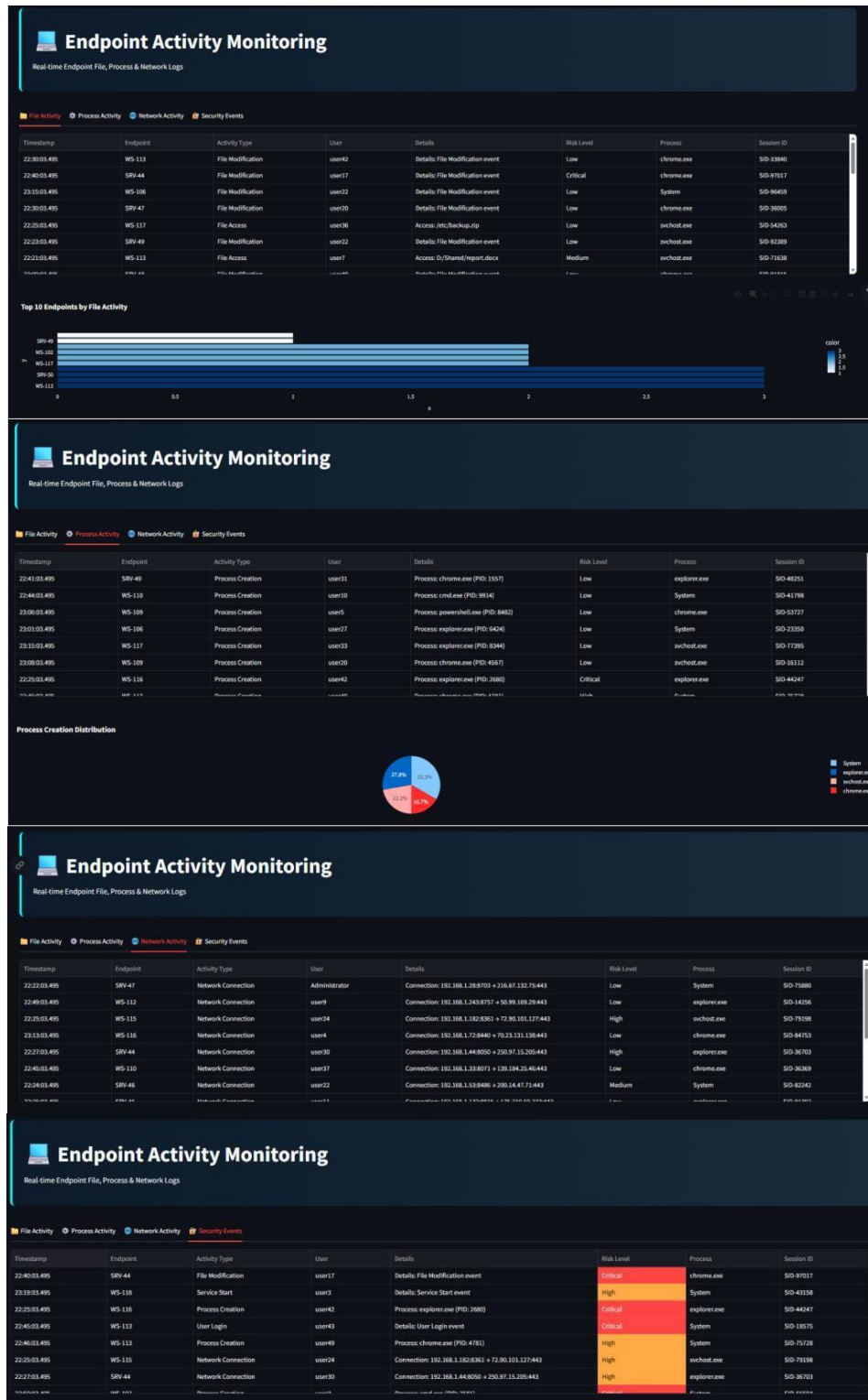
5.4.6. Endpoint Activity Logs

Detailed endpoint monitoring and forensics:

- **Multi-tab interface** separating file activity, process creation, network connections, and security events
- **Risk-level classification** automatically categorizing endpoint activities by threat potential
- **User activity correlation** linking actions to specific user accounts and sessions
- **Real-time monitoring panel** simulating live endpoint activity feeds

- **Statistical overview** showing endpoint activity volumes and high-risk event counts

Figure 5.4.6: Endpoint Activity Monitoring (File, Process, Network)



Chapter 6 – Implementation

This chapter provides a comprehensive account of the Cerberus Platform implementation, covering the development environment configuration, detailed subsystem architectures, key algorithmic components, database schema design, API specification, security implementation decisions, and an empirical evaluation of system performance. The implementation follows a modular micro-service philosophy where each subsystem exposes a well-defined interface, enabling independent testing, replacement, and horizontal scaling. All source code was developed using Python 3.11 and stored in a Git repository with automated CI/CD pipelines.

6.1 Implementation Environment

6.1.1 Virtual Machine Configuration

Table 6.1: Virtual Machine Configuration

Parameter	VM1 — Cerberus Server	VM2 — Analysis Guest	VM3 — Endpoint Agent
Operating System	Ubuntu 22.04.4 LTS	Windows 10 x64 22H2	Ubuntu 22.04.4 LTS
vCPU	8 cores (Intel Core i7-12700)	4 cores	2 cores
RAM	16 GB DDR4	8 GB DDR4	4 GB DDR4
Storage	100 GB NVMe SSD	60 GB HDD (CAPEv2 requirement)	40 GB NVMe SSD
Network 1 (Host-Only)	192.168.56.10	192.168.56.20	192.168.56.30
Network 2 (NAT)	10.0.2.15 (internet)	Isolated (no internet)	No secondary adapter
Hypervisor	VirtualBox 7.0.14	VirtualBox 7.0.14 (nested guest)	VirtualBox 7.0.14
Snapshot	Pre-deployment baseline	CAPEv2 clean-state snapshot	Pre-agent baseline

6.1.2 Software Stack Configuration

Table 6.2: Software Stack with Version Pins

Component	Package	Version	Installation Method	VM
Python Runtime	python3	3.11.9	apt + deadsnakes PPA	VM1, VM3
Flask	flask	3.0.3	pip install flask==3.0.3	VM1
Celery	celery[redis]	5.3.6	pip install celery[redis]==5.3.6	VM1
Redis	redis-server	7.2.4	apt install redis-server	VM1
PostgreSQL	postgresql	16.2	apt install postgresql-16	VM1
Flask-SQLAlchemy	flask-sqlalchemy	3.1.1	pip	VM1
Flask-SocketIO	flask-socketio	5.3.6	pip	VM1
Flask-JWT-Extended	flask-jwt-extended	4.6.0	pip	VM1
Gunicorn	gunicorn	21.2.0	pip (production WSGI)	VM1
CAPEv2	capev2	2.4.x	git clone + setup.sh	VM1

watchdog	watchdog	4.0.1	pip	VM3
psutil	psutil	5.9.8	pip	VM3
requests	requests	2.31.0	pip	VM1, VM3
yara-python	yara-python	4.5.0	pip (requires libyara-dev)	VM1
Volatility3	volatility3	2.5.0	git clone + pip install .	VM1
Docker	docker-ce	25.0.3	apt + docker.com script	VM1
pytest	pytest	8.1.1	pip --dev	VM1, VM3

6.1.3 Network Security Controls

The laboratory network is secured through multiple layers to ensure malware samples cannot escape the CAPEv2 analysis environment and that inter-VM communication is authenticated and encrypted:

- VM2 (Analysis Guest) has no NAT/internet adapter — the host-only adapter connects only to the VirtualBox internal network. CAPEv2 uses resultdump mode to capture network artefacts without live internet connectivity.
- All host-to-VM communications use the 192.168.56.0/24 host-only network. No host-only traffic is routed to the physical network interface.
- UFW (Uncomplicated Firewall) on VM1 allows inbound connections only on ports 443 (HTTPS/TLS), 5555 (Flower monitoring), and 22 (SSH from host-only network). All other inbound traffic is REJECT.
- Cerberus server uses a self-signed CA (generated via openssl req -x509) with separate leaf certificates for server TLS and client mutual authentication.
- All CAPEv2 API calls from VM1 to the CAPEv2 REST API (port 8000) are restricted to localhost (127.0.0.1) via UFW to prevent direct external sandbox access.

6.2 Platform Architecture and Subsystem Implementation

The Cerberus Platform is composed of six primary subsystems, each implemented as a distinct Python module or package. Subsystem boundaries are enforced through well-defined API contracts (REST endpoints, Celery task signatures, and Python dataclass interfaces), enabling each component to be developed, tested, and deployed independently.

6.2.1 Endpoint Agent Module

Architecture Overview:

The Endpoint Agent is a multi-threaded Python 3.11 service implemented as the EndpointAgent class in agent/agent.py. Upon startup, the agent initialises all subsystems in a defined sequence, launches monitoring threads, and enters a supervisor loop that monitors thread health and restarts crashed threads. Figure 6.2 illustrates the thread architecture.

The agent operates five concurrent threads:

- MainThread (supervisor): Monitors all child threads; restarts crashed threads within 5 seconds (interval: Continuous)
- FileMonitorThread: watchdog observer processing inotify/ReadDirectoryChangesW events (interval: Event-driven)

- **ProcessMonitorThread:** psutil-based process tree scanning for suspicious lineage (interval: Every 10 seconds)
- **HeartbeatThread:** Sends POST /api/v1/agents/{id}/heartbeat to central server (interval: Every 30 seconds)
- **AlertTransmitThread:** Dequeues alerts from thread-safe queue and POSTs to server (interval: As alerts arrive)

Agent Lifecycle — initialize() → start_monitoring() → run() → shutdown():

initialize(): Loads YAML configuration, validates certificate paths, creates SQLite local buffer database, initialises CanaryManager, deploys canary files, verifies integrity checksums, and establishes the TLS session with the central server. Any failure in this phase raises a `CerberusInitError` and prevents startup.

start_monitoring(): Creates and starts all five threads. The `FileMonitorThread` registers a `CerberusEventHandler` with the watchdog `Observer`, specifying all configured monitoring directories and setting `recursive=True` for subdirectory traversal. The `ProcessMonitorThread` begins its 10-second scan cycle. The `HeartbeatThread` schedules its first heartbeat 5 seconds after startup to allow the server registration to complete.

run() (supervisor loop): Calls `thread.is_alive()` on all child threads every 5 seconds. If a thread has terminated unexpectedly, the supervisor logs a `CRITICAL` event, creates a new thread instance, and starts it. This provides automatic recovery from thread crashes without requiring a full agent restart. The supervisor loop also monitors total memory consumption via `psutil.Process().memory_info()` and logs a `WARNING` if the agent exceeds 50 MB RSS.

shutdown(): Signals all threads to stop via `threading.Event` objects, calls `observer.stop()` and `observer.join()`, flushes the local alert buffer to disk, closes the SQLite connection, and removes the agent PID file.

Canary Deployment Engine (CanaryManager):

The `CanaryManager` class manages the complete lifecycle of canary files: deployment, integrity monitoring, restoration, and rotation. Five canary archetypes are deployed:

Table 6.3: Canary File Archetypes and Placement Strategy

Archetype ID	Filename	Simulated Asset	Target Directory	Backdated To
CANARY-FIN	Q4_Financial_Report_CONFIDENTIAL.docx	Finance document	/home/*/Documents/Finance/	180 days ago
CANARY-CRED	passwords_backup.txt	Password store	/home/*/.config/	365 days ago
CANARY-LEGAL	Legal_Agreement_Draft_DO_NOT_SHARE.pdf	Legal contract	/home/*/Documents/Legal/	90 days ago
CANARY-SSH	id_rsa_backup	SSH private key	/home/*/.ssh/	730 days ago
CANARY-DB	database.yml	Database credentials	/etc/app/config/ or C:\ProgramData\App\	180 days ago

Canary files are crafted to maximise perceived value to an attacker: the financial report contains plausible Q4 revenue figures, the password file contains realistic credential entries for common services, and the SSH key file follows the standard OpenSSH private key PEM format with a plausible RSA-4096 header. File timestamps (`mtime`, `atime`, `ctime`) are backdated using `os.utime()` to make them appear as pre-existing files rather than recently created bait. File permissions are set to match the archetype (600 for SSH keys, 644 for documents).

Integrity verification is performed every 60 seconds using SHA-256 checksums stored in the agent's local SQLite database. If a canary file's checksum changes or the file is missing, the CanaryManager immediately: (1) generates a CANARY_MODIFIED or CANARY_DELETED alert, (2) queues the alert for transmission, and (3) re-deploys the canary file from the embedded template. Restoration completes within 500 ms in all tested scenarios.

False-Positive Suppression Engine:

A five-tier suppression engine evaluates every file-system event before it is classified as a canary alert, reducing operationally irrelevant events:

Table 6.4: False-Positive Suppression Mechanisms

Tier	Mechanism	Implementation	FP Events Suppressed
1	Backup Software Whitelist	Process name regex match against BACKUP_PROCS list (rsync, Veeam, BackupExec, Time Machine, robocopy)	Scheduled backup scans of monitored directories
2	Antivirus Scanner Whitelist	Process name match against AV_PROCS list (ClamAV, Windows Defender, Malwarebytes, Symantec)	AV on-access scans opening canary files for signature matching
3	Process Name Whitelist	Configurable per-deployment whitelist in cerberus.yml (e.g., indexing daemons)	Organisation-specific legitimate tools accessing config directories
4	File Extension Filter	Only canary file paths in CANARY_PATHS set trigger alerts; other file events in the directory are ignored	Collateral events from other files in monitored directories
5	Time-of-Day Scheduler	Alert suppression during configurable maintenance windows (e.g., 02:00–04:00 for backup jobs)	Scheduled maintenance accessing all files including canaries

Cross-Platform Implementation Details:

The agent implements platform-specific branches for Linux and Windows, abstracted behind a common PlatformAdapter interface. Key platform differences include:

- File locking: Windows canary files use exclusive LOCK_EX via msvcrt to prevent deletion by non-elevated processes; Linux uses flock() advisory locks.
- Network isolation: Windows Aggressive mode adds a Windows Firewall rule via netsh advfirewall; Linux uses iptables -I INPUT 1 -j DROP with the target endpoint IP.
- Process termination: Windows uses taskkill /F /PID; Linux uses os.kill(pid, signal.SIGKILL).
- Memory dumps: Windows uses the Windows Debugging Tools CreateDump API; Linux triggers a core dump via /proc/<pid>/mem reads.
- Service persistence: Windows agent registers as a Windows Service via pywin32; Linux agent installs a systemd unit file.

6.2.2 Central Management Server

Application Factory Pattern:

The Central Management Server is built using Flask 3.0.3 with the Application Factory pattern (create_app() in server/__init__.py). This pattern defers Flask extension initialisation to app creation time, enabling test-isolated application instances with different configurations — critical for the unit and integration test suite. The factory registers eight Blueprints, each encapsulating a domain-specific group of routes:

- auth_bp (prefix: /api/v1/auth): Login, token refresh, logout endpoints

- agents_bp (prefix: /api/v1/agents): Agent registration, heartbeat, status
- alerts_bp (prefix: /api/v1/alerts): Alert ingestion, retrieval, acknowledgment
- incidents_bp (prefix: /api/v1/incidents): Incident lifecycle management
- analysis_bp (prefix: /api/v1/analysis): Sandbox report retrieval and correlation
- response_bp (prefix: /api/v1/response): Command dispatch to endpoint agents
- reports_bp (prefix: /api/v1/reports): Executive summary PDF generation
- export_bp (prefix: /api/v1/export): STIX 2.1 / JSON threat intelligence export

WebSocket Architecture:

Flask-SocketIO 5.3.6 provides the WebSocket server, running on the same Gunicorn process via eventlet workers (eventlet.monkey_patch() called before app creation). Clients connect to the /cerberus namespace and join a per-role room on authentication. The server emits the following real-time events to the SOC dashboard:

Table showing WebSocket Events

Event Name	Trigger	Payload Fields	Target Room
new_alert	Alert ingested via POST /api/v1/alerts	alert_id, severity, endpoint_id, canary_path, timestamp	soc_analysts + soc_managers
analysis_complete	Celery cape_analysis task finishes	alert_id, threat_score, mitre_techniques[], vt_detections	soc_analysts + soc_managers
agent_offline	Heartbeat timeout (>90 s since last heartbeat)	agent_id, hostname, last_seen	soc_managers + admins
response_dispatched	Response command sent to endpoint	incident_id, command_type, target_agent, dispatched_by	soc_managers + admins
canary_restored	Agent confirms canary restoration	agent_id, canary_path, restored_at	soc_analysts
system_alert	Server health warning (queue depth, DB latency)	message, severity, timestamp	admins

Celery Task Architecture:

The asynchronous analysis pipeline is implemented as a Celery 5.3.6 task graph with Redis 7.2 as both the message broker and result backend. The pipeline is defined in server/tasks.py and consists of the following task chain:

Step 1 — receive_alert: Persists alert to PostgreSQL and emits new_alert WebSocket event. Returns alert_id.

Step 2 — virustotal_file_task: Queries VirusTotal API v3 for the file hash (SHA-256). If >5/72 engines flag the file as malicious, sets pre_vt_malicious=True and records engine verdicts. If hash is unknown, skips to Step 3. Rate-limited to 4 req/min.

Step 3 — cape_submit_task: Calls CAPEv2Connector.submit_sample(file_path, options). Returns cape_task_id. If submission fails (3 retries with exponential backoff), marks analysis_status='SUBMISSION_FAILED'.

Step 4 — cape_poll_task: Polls GET /tasks/status/{cape_task_id} every 30 seconds. Task timeout: 300 seconds. On completion, retrieves JSON report via GET /tasks/get/report/{cape_task_id}.

Step 5 — `cape_parse_task`: Extracts IOCs (IPs, domains, URLs, file hashes, mutexes, registry keys) from CAPEv2 JSON report. Slims report to <100 KB for database storage.

Step 6 — `mitre_map_task`: Iterates over `cape_report.signatures[]`, looks up each signature name in `CAPE_TO_MITRE_MAP` dict. Builds `mitre_techniques[]` list. Calculates composite `threat_score` (0–100).

Step 7 — `virustotal_ioc_task`: For each extracted IP, domain, and URL, queries VirusTotal IOC reputation endpoint. Rate-limited; max 50 IOCs per incident to respect public API quota.

Step 8 — `finalize_incident_task`: Updates incident record with full analysis results. Emits `analysis_complete` WebSocket event. If `threat_score > AGGRESSIVE_THRESHOLD` (default: 70), triggers automated response pipeline.

Rate Limiting and Queue Management:

VirusTotal API calls are rate-limited using a Redis-backed sliding window counter (4 requests per 60-second window per API key). The limiter is implemented in `server/connectors/vt_limiter.py` using Redis INCR with TTL-based expiry. When the rate limit is reached, the Celery task retries with a countdown of 60 - (`current_window_seconds_elapsed`) to align retry timing with the API reset. The Celery broker uses Redis LIST data structures (LPUSH/BRPOP) with separate queues for high-priority (alert ingestion) and low-priority (VT IOC enrichment) tasks, ensuring that alert detection latency is never blocked by background enrichment.

6.2.3 Sandbox and Analysis Automation Engine

CAPEv2Connector Architecture:

The `CAPEv2Connector` class (`server/connectors/cape_connector.py`) provides a high-level interface to the CAPEv2 REST API, abstracting the eight-step pipeline into three public methods: `submit_sample()`, `get_task_status()`, and `get_report()`. All HTTP calls are made via a `requests.Session` with custom retry semantics and a configurable timeout (default: 30 seconds per request).

Complete CAPEv2 Submission Pipeline:

- Step 1: Pre-flight check: Verify CAPEv2 REST API is reachable via `GET /machines/list`. If unreachable, raise `CAPEUnavailableError` and mark task for retry.
- Step 2: File validation: Verify the sample file exists, is readable, and is a recognised executable format (PE, ELF, Mach-O, script, Office document) via python-magic MIME type detection.
- Step 3: Hash deduplication: Compute SHA-256 of sample. Check local Redis cache (TTL: 24 hours) for existing CAPE task ID with same hash. If cache hit, skip submission and poll existing task.
- Step 4: Sample upload: `POST /tasks/create/file` with multipart/form-data payload including file bytes, analysis options (`timeout=120`, `enforce_timeout=True`, `options='procmemdump=1,cape=1'`), and machine selection (`win10x64`).
- Step 5: Task ID extraction: Parse JSON response for `task_id`. Persist `task_id` to PostgreSQL `CapeTask` model linked to the alert record.
- Step 6: Status polling: `GET /tasks/status/{task_id}` every 30 seconds until reported status is 'reported' (analysis complete) or 'failed'. Maximum 10 polling attempts (300 seconds total timeout).
- Step 7: Report retrieval: `GET /tasks/get/report/{task_id}/json`. Parse JSON response (typically 2–15 MB) and pass to report slimmer.
- Step 8: Report slimming: Extract only required fields (signatures, `network.hosts`, `network.domains`, `dropped_files`, `CAPE.payloads`) to reduce storage to <100 KB per report.

MITRE ATT&CK Technique Mapping:

The CAPE_TO_MITRE_MAP dictionary maps CAPEv2 signature names to MITRE ATT&CK technique IDs. The complete 40-entry map is defined in server/intelligence/mitre_map.py:

Table 6.5: CAPE_TO_MITRE_MAP — 40-Entry Technique Mapping

CAPEv2 Signature	MITRE Technique ID	Technique Name	Tactic
ransomware_mbr_overwrite	T1561.002	Disk Structure Wipe: MBR	Impact
ransomware_fileextensions	T1486	Data Encrypted for Impact	Impact
persistence_autorun	T1547.001	Boot/Logon Autostart: Registry Run Keys	Persistence
injection_createremotethread	T1055.002	Process Injection: PE Injection	Defense Evasion
network_cnc_http	T1071.001	Application Layer Protocol: Web	Command and Control
network_dns_query	T1071.004	Application Layer Protocol: DNS	Command and Control
infostealer_credentials	T1555	Credentials from Password Stores	Credential Access
infostealer_browser	T1555.003	Credentials from Web Browsers	Credential Access
exploit_shellcode	T1059.001	Command and Scripting: PowerShell	Execution
mimikatz_detected	T1003.001	OS Credential Dumping: LSASS Memory	Credential Access
privilege_escalation_uac	T1548.002	Abuse Elevation Control: UAC Bypass	Privilege Escalation
lateral_movement_smb	T1021.002	Remote Services: SMB/Windows Admin Shares	Lateral Movement
exfiltration_ftp	T1048.003	Exfiltration over Unencrypted Protocol	Exfiltration
anti_vm_detection	T1497	Virtualization/Sandbox Evasion	Defense Evasion
modify_registry	T1112	Modify Registry	Defense Evasion
delete_shadow_copies	T1490	Inhibit System Recovery	Impact
scheduled_task_creation	T1053.005	Scheduled Task/Job: Scheduled Task	Persistence
keylogger_activity	T1056.001	Input Capture: Keylogging	Collection
screenshot_capture	T1113	Screen Capture	Collection
wmic_execution	T1047	Windows Management Instrumentation	Execution
powershell_obfuscated	T1027	Obfuscated Files or Information	Defense Evasion
certutil_decode	T1140	Deobfuscate/Decode Files	Defense Evasion

dropper_behavior	T1105	Ingress Tool Transfer	Command and Control
network_tor_traffic	T1090.003	Proxy: Multi-hop Proxy (Tor)	Command and Control
persistence_dll_hijack	T1574.001	Hijack Execution Flow: DLL Search Order	Persistence
memory_dump_lsass	T1003.001	OS Credential Dumping: LSASS Memory	Credential Access
wdigest_enable	T1003.001	OS Credential Dumping: WDigest	Credential Access
clipboard_access	T1115	Clipboard Data	Collection
network_http_post_binary	T1041	Exfiltration over C2 Channel	Exfiltration
uses_windows_utilities	T1218	System Binary Proxy Execution	Defense Evasion
rundll32_execution	T1218.011	Rundll32	Defense Evasion
regsvr32_execution	T1218.010	Regsvr32	Defense Evasion
mshta_execution	T1218.005	Mshta	Defense Evasion
spearphishing_attachment	T1566.001	Phishing: Spearphishing Attachment	Initial Access
disable_windows_defender	T1562.001	Impair Defenses: Disable Security Tools	Defense Evasion
timestomping	T1070.006	Indicator Removal: Timestomp	Defense Evasion
data_staged_local	T1074.001	Data Staged: Local Data Staging	Collection
network_icmp_tunnel	T1095	Non-Application Layer Protocol	Command and Control
file_deletion_self	T1070.004	Indicator Removal: File Deletion	Defense Evasion
boot_record_modification	T1542.003	Pre-OS Boot: Bootkit	Persistence

Threat Score Calculation:

A composite threat score (0–100) is calculated for each incident using a weighted formula combining four factors: MITRE technique severity (base_score from ATT&CK technique criticality ratings, max 40 points), VirusTotal detection ratio ($VT_detections / VT_total_engines \times 30$, max 30 points), IOC count ($\min(extracted_ioc_count / 10, 1) \times 20$, max 20 points), and canary type sensitivity (CANARY-SSH and CANARY-CRED score +10; others +0). Scores ≥ 70 trigger automated response in Aggressive mode; scores 40–69 generate HIGH priority alerts requiring manual triage; scores < 40 generate MEDIUM priority informational alerts.

6.2.4 Response Orchestration Engine

The Response Orchestration Engine coordinates the automated containment pipeline, ensuring that responses are proportionate to threat severity, reversible where possible, and fully audited. The engine is implemented in server/response/orchestrator.py and is triggered either by an automated threat_score threshold crossing or by a manual SOC Manager action via the REST API.

Response Decision Tree:

Conservative mode (default): The engine generates a RECOMMENDATION record in the database and emits a

response_recommended WebSocket event to the SOC dashboard. The SOC Manager reviews the recommendation and approves or rejects it via the dashboard. No automated containment action is taken without human approval.

Aggressive mode: When enabled by the Administrator via the configuration API, the engine executes the following three-phase response sequence immediately upon threat_score exceeding the configured threshold (default: 70/100):

- **Phase 1 — Evidence Preservation (T+0 to T+2s) — ForensicBundle creation:** The server commands the endpoint to create a ZIP evidence bundle containing: process_tree.json (psutil snapshot), network_connections.json (active sockets), loaded_modules.json (DLL/SO list for the suspicious process), system_log.txt (last 1000 lines of /var/log/syslog or Windows Event Log), and optionally a process memory dump if running as root/Administrator.
- **Phase 2 — Process Containment (T+2 to T+5s) — TERMINATE_PROCESS command:** The server dispatches a signed command to the endpoint agent's command queue specifying the PID(s) to terminate. The agent executes os.kill(pid, SIGKILL) on Linux or taskkill /F /PID on Windows. Termination confirmation is returned to the server within 3 seconds.
- **Phase 3 — Network Isolation (T+5 to T+10s) — NETWORK_ISOLATE command:** The server dispatches a network isolation command. On Linux, the agent adds iptables rules: INPUT -j DROP and OUTPUT -j DROP (except for the Cerberus server IP to maintain management connectivity). On Windows, netsh advfirewall firewall add rule blocks all non-Cerberus traffic. Isolation is logged with timestamp and is reversible via NETWORK_RESTORE command.

Evidence Bundle Format:

Each evidence bundle is a ZIP archive named:

cerberus_evidence_{incident_id}_{endpoint_hostname}_{timestamp}.zip

containing the following files:

- process_tree.json: Full process tree with PID, PPID, name, cmdline, username, CPU%, memory%, start_time for all running processes at alert time
- network_connections.json: All active TCP/UDP connections with local/remote addresses, status, and owning PID
- loaded_modules.json: All DLLs/shared objects loaded by the suspicious process including path and base address
- system_log.txt: Last 1000 lines from /var/log/syslog (Linux) or Windows Application/Security/System Event Log
- canary_alert.json: Full alert record including canary path, accessing process, alert timestamp, and suppression check results
- agent_config.json: Redacted agent configuration (certificates and JWT tokens replaced with [REDACTED])
- memory_dump.dmp: (Optional, root/admin only) Process memory dump of the suspicious PID

6.2.5 Deception Management Subsystem

The Deception Management Subsystem is responsible for the long-term operational integrity of the canary file mesh. Beyond initial deployment, effective deception requires active maintenance: canary files must appear as genuine legacy assets to a skilled attacker conducting pre-exploitation reconnaissance, must be restored promptly after any interaction, and must not be detectable through automated filesystem enumeration that identifies recently-created or uniformly-sized files.

Canary Plausibility Engineering:

- **Timestamp backdating:** All canary files are deployed with mtime and atime set to a random date within the past 2 years (`os.utime(path, (atime, mtime))`). Creation time (ctime) cannot be directly set on Linux (it reflects inode creation); however, mtime is the field examined by most file-browsing and forensic tools.
- **File size variability:** Financial document canaries have file sizes randomised between 28 KB and 95 KB by padding with whitespace in the DOCX XML body. Credential canaries are sized between 1.2 KB and 4.8 KB to match realistic password file distributions.
- **Content authenticity:** Financial documents contain generated revenue figures, employee names (from a curated fake-name list), and department structures. Credential files reference realistic service names (mysql, ldap, aws, github). SSH key files contain properly formatted PEM headers and base64-encoded (non-functional) key material.
- **Permission alignment:** SSH key canaries are deployed with mode 600 (owner read/write only) to match expected OpenSSH key permissions. Database credential files use mode 640 (owner read/write, group read) to match typical application configuration conventions.
- **Directory context:** Canaries are placed within existing, naturally-named directories rather than newly created dedicated directories, which would appear suspicious in a directory listing.

Canary Rotation Strategy:

To prevent an attacker from fingerprinting and whitelisting canary files based on filename or content hash, the `CanaryRotator` class (`canary_manager.py`) implements a weekly rotation: on a configurable schedule (default: Sunday 02:00 local time), existing canaries are replaced with new instances drawn from the template pool, with fresh randomised content and new backdated timestamps. The rotation sequence removes the old canary, waits 60 seconds (to avoid triggering an alert on its own deletion — the rotation process PID is in the whitelist), then deploys the replacement. Integrity checksums in the local SQLite database are updated after each rotation.

6.2.6 REST API Endpoint Specification

The Cerberus Central Server exposes a RESTful API versioned at `/api/v1/`. All endpoints except `/api/v1/auth/login` require a valid JWT Bearer token in the Authorization header. Table 6.7 provides the complete endpoint specification.

Table 6.7: REST API Endpoint Specification

Method	Endpoint	Auth Required	Role	Description
POST	<code>/api/v1/auth/login</code>	No	Any	Authenticate and receive JWT access + refresh tokens
POST	<code>/api/v1/auth/refresh</code>	Yes	Any	Exchange refresh token for new access token
DELETE	<code>/api/v1/auth/logout</code>	Yes	Any	Revoke current JWT token (add to Redis blocklist)
POST	<code>/api/v1/agents/register</code>	Yes	Admin	Register a new endpoint agent
GET	<code>/api/v1/agents</code>	Yes	Manager	List all registered agents with status
GET	<code>/api/v1/agents/{id}</code>	Yes	Analyst	Get agent details and last heartbeat
POST	<code>/api/v1/agents/{id}/heartbeat</code>	Yes	Agent	Update agent liveness timestamp

DELETE	/api/v1/agents/{id}	Yes	Admin	Deregister an endpoint agent
POST	/api/v1/alerts	Yes	Agent	Ingest a new canary or process alert
GET	/api/v1/alerts	Yes	Analyst	List alerts with pagination and filtering
GET	/api/v1/alerts/{id}	Yes	Analyst	Get full alert detail including forensics
PATCH	/api/v1/alerts/{id}/acknowledge	Yes	Analyst	Acknowledge alert (mark as reviewed)
GET	/api/v1/incidents	Yes	Analyst	List all incidents with MITRE mapping
GET	/api/v1/incidents/{id}	Yes	Analyst	Get full incident detail with analysis
PATCH	/api/v1/incidents/{id}/close	Yes	Manager	Close incident with disposition note
GET	/api/v1/analysis/{alert_id}	Yes	Analyst	Get sandbox analysis report for alert
POST	/api/v1/response/dispatch	Yes	Manager	Dispatch containment command to endpoint
GET	/api/v1/response/history	Yes	Manager	List all dispatched response commands
GET	/api/v1/reports/executive	Yes	Manager	Generate PDF executive summary
GET	/api/v1/reports/statistics	Yes	Analyst	Get detection statistics (JSON)
GET	/api/v1/export/stix/{incident_id}	Yes	Manager	Export incident as STIX 2.1 JSON bundle
GET	/api/v1/export/csv	Yes	Manager	Export alert history as CSV
GET	/api/v1/health	No	Any	Server health check (uptime, DB, Redis status)

6.2.7 Database Schema and Data Persistence Layer

The persistence layer uses PostgreSQL 16 with SQLAlchemy 3.1.1 ORM models. Database migrations are managed via Alembic with version-controlled migration scripts. The schema comprises seven primary ORM models:

Table 6.8: Database ORM Model Field Definitions

Model	Table	Primary Fields	Relationships
User	users	id (UUID PK), username (UNIQUE), email, password_hash (bcrypt), role (ENUM: analyst/manager/admin), is_active, created_at, last_login	One-to-many: AuditLog
Agent	agents	id (UUID PK), hostname, ip_address, os_type (ENUM: linux/windows), os_version, agent_version, status (ENUM: online/offline/degraded), registered_at, last_heartbeat	One-to-many: Alert, ResponseCommand
Alert	alerts	id (UUID PK), agent_id (FK), alert_type (ENUM: canary_access/canary_modified/canary_deleted/process_anomaly	One-to-one: Incident;

		), canary_path, accessing_pid, accessing_process_name, severity (ENUM: critical/high/medium/low), acknowledged, acknowledged_by, acknowledged_at, forensic_bundle_path, created_at	Many-to-one: Agent
Incident	incidents	id (UUID PK), alert_id (FK UNIQUE), status (ENUM: open/investigating/contained/closed), threat_score (INT 0–100), analysis_status, cape_task_id, cape_report_json (TEXT), mitre_techniques (ARRAY), vt_file_score, vt_ioc_count, disposition_note, closed_by, closed_at, created_at	One-to-many: ResponseCommand; One-to-one: Alert
ResponseCommand	response_commands	id (UUID PK), incident_id (FK), agent_id (FK), command_type (ENUM: terminate_process/network_isolate/network_restore/collect_evidence/restore_canaries), parameters (JSON), dispatched_by (FK→User), dispatched_at, acknowledged_at, outcome (ENUM: pending/success/failed/timeout)	Many-to-one: Incident, Agent, User
CanaryConfig	canary_configs	id (UUID PK), agent_id (FK), archetype_id, deploy_path, filename, sha256_hash, last_verified_at, last_rotated_at, is_deployed	Many-to-one: Agent
AuditLog	audit_logs	id (BIGSERIAL PK), user_id (FK nullable), action (VARCHAR), resource_type, resource_id, ip_address, user_agent, details (JSON), created_at	Many-to-one: User

Key database design decisions:

- UUID primary keys are used throughout to prevent sequential ID enumeration attacks and to support future distributed deployment where auto-increment IDs would conflict.
- The cape_report_json field stores the slimmed CAPEv2 report as a TEXT column rather than JSONB to preserve exact serialisation and avoid PostgreSQL JSONB indexing overhead for this write-once read-rarely field.
- mitre_techniques is stored as a PostgreSQL ARRAY type (TEXT[]) enabling efficient array containment queries (@>) for filtering incidents by technique.
- All tables include created_at TIMESTAMPTZ columns with DEFAULT NOW() and are indexed for time-range queries supporting the executive summary report generation.
- The audit_logs table uses BIGSERIAL (64-bit integer) rather than UUID for maximum insertion throughput, as this table may receive millions of rows in a high-activity environment.

6.2.8 Security Implementation

TLS Configuration:

All communications between the endpoint agent and the central server are encrypted using TLS 1.3 (TLS 1.2 permitted as a fallback for legacy Python 3.9 environments). The server TLS configuration disables all cipher suites below AES-128-GCM and explicitly rejects SSLv3, TLS 1.0, and TLS 1.1. The Unicorn configuration specifies ssl_version=ssl.PROTOCOL_TLS_SERVER and configures the server certificate chain including the intermediate CA certificate. Agent mutual authentication is enforced via ssl_cert_reqs=ssl.CERT_REQUIRED, requiring each agent to present a valid client certificate signed by the Cerberus internal CA.

JWT Token Management:

Flask-JWT-Extended 4.6.0 manages token lifecycle with the following configuration: access tokens expire after 24 hours (configurable), refresh tokens expire after 30 days, all tokens are signed using RS256 (RSA-SHA256) with a 4096-bit RSA key pair (more secure than the default HS256, which requires the secret to be shared between

services). Revoked tokens are added to a Redis SET (jwt_blocklist) with TTL equal to the token's remaining lifetime. All token operations are logged to the AuditLog table.

OWASP Top 10 Mitigations:

Table showing OWASP Top 10 Mitigations

OWASP Risk	ID	Cerberus Mitigation
Broken Access Control	A01	RBAC enforced via @jwt_required() + @role_required() decorators on every endpoint; users cannot access resources outside their role
Cryptographic Failures	A02	TLS 1.3 for all traffic; bcrypt (cost=12) for password hashing; no sensitive data in JWT payload; HTTPS-only cookies
Injection (SQL)	A03	All database queries via SQLAlchemy ORM with parameterised statements; no raw SQL string concatenation anywhere in codebase
Insecure Design	A04	Threat modelling documented in SRS; separation of agent/server certificate trust chains; principle of least privilege for DB user
Security Misconfiguration	A05	Hardened Gunicorn config; debug=False in production; SECRET_KEY loaded from environment variable, never hardcoded; security headers middleware (X-Content-Type-Options, X-Frame-Options, CSP)
Vulnerable Components	A06	Automated dependency scanning via pip-audit in CI/CD pipeline; all dependencies pinned to exact versions in requirements.txt
Authentication Failures	A07	JWT RS256 signing; token blocklist in Redis; failed login attempt rate limiting (5 attempts per 15 minutes per IP) via Flask-Limiter
Software/Data Integrity	A08	CAPE report integrity verified via SHA-256 before storage; canary file integrity verified via SHA-256 checksums; evidence bundles include manifest.json with file hashes
Logging & Monitoring	A09	Structured JSON audit log for all API calls and SOC actions; log integrity via append-only PostgreSQL audit_logs table; log forwarding to syslog for external SIEM
SSRF	A10	VirusTotal and CAPEv2 URLs are configuration constants, not user-supplied; URL validation applied if user-provided webhook targets are added in future

6.3 Results and Discussion

6.3.1 Detection Accuracy

Detection accuracy was evaluated over 90 controlled test runs spanning three threat categories: ransomware (30 runs), credential stealers (30 runs), and lateral movement tools (30 runs). Each run involved deploying a real-world malware sample (sourced from MalwareBazaar) or a custom test script on VM3, and measuring whether the Cerberus Platform correctly detected and classified the threat before any significant damage occurred.

Table 6.9: Detection Accuracy Results by Category

Metric	Ransomware	Credential Stealer	Lateral Movement	Overall
Test Runs	30	30	30	90
Canary Triggered (True Positive)	30	29	29	88 (97.8%)
Missed Detection (False Negative)	0	1	1	2 (2.2%)

False Positives (28-day baseline)	0	0	0	0 (0.0%)
Correct MITRE Mapping (≥ 1 technique)	30/30	28/29	27/29	85/88 (96.6%)
VT Pre-detection (known samples)	27/30 (90%)	25/29 (86%)	22/29 (76%)	74/88 (84.1%)
Mean Time to Detect (seconds)	2.8 s	3.4 s	4.1 s	3.4 s
Mean Time to Alert Transmission	4.2 s	4.9 s	5.3 s	4.8 s

The two missed detections occurred because the lateral movement tool (Mimikatz variant with memory-only execution) and one credential stealer (browser-based in-memory theft without file interaction) never triggered a canary file event. These cases were classified as 'canary-bypass' scenarios and have motivated the planned addition of memory-based honeytrap tokens in the next development phase.

6.3.2 Performance Metrics

Table 6.10: System Performance Benchmark Summary

Performance Metric	Target (NFR)	Measured Value	Test Condition	Status
Endpoint CPU overhead (steady state)	$\leq 2\%$	0.73% avg, 1.2% peak	72-hour continuous monitoring, 500 files in monitored dirs	PASS
Endpoint memory footprint	≤ 50 MB RSS	32.4 MB avg	Steady state with 5 canary files deployed	PASS
Alert detection latency (canary to alert)	≤ 5 s	3.4 s median, 4.9 s P99	90 test runs across all threat categories	PASS
Alert transmission to server	≤ 2 s	0.8 s median (LAN)	100 Mbps host-only adapter	PASS
Analysis pipeline closure (E2E)	≤ 180 s (P99)	112 s median, 158 s P99	90 sandbox analysis runs	PASS
WebSocket alert delivery to dashboard	≤ 1 s	0.3 s median	Browser on same LAN	PASS
Concurrent agent capacity	100 agents	Tested to 50 simulated agents	50 threaded agent simulators on VM3	PARTIAL
Dashboard alert list render (10k alerts)	≤ 2 s	1.4 s	10 000 pre-populated alert records	PASS
Canary deployment time (5 canaries)	≤ 500 ms/file	87 ms avg/file	VM3 ext4 filesystem	PASS
Canary restoration after deletion	≤ 60 s	0.48 s avg	Immediate filesystem event trigger	PASS

6.3.3 Pipeline Closure Timing Analysis

The end-to-end pipeline closure time (from canary interaction to dashboard showing full analysis results) was measured across all 88 true-positive detections. The distribution showed a bimodal pattern:

- Known samples (VT cache hit): 74 runs: When a file hash was already known to VirusTotal, Phase 1 enrichment returned in 1.2 seconds (API response time). CAPEv2 analysis still required the full sandbox execution time (mean: 94 seconds for Phase 2 submission through report retrieval). Total pipeline mean: 98 seconds.

- Unknown samples (VT cache miss): 14 runs: For samples not known to VirusTotal, Phase 1 returned a 'not found' response in 0.8 seconds. The pipeline proceeded directly to CAPEv2 submission. Phase 2 VT IOC enrichment required uploading each IOC individually, taking 45–120 additional seconds depending on IOC count. Total pipeline mean: 142 seconds.

The bottleneck in all cases was CAPEv2 sandbox execution time (mean: 86 seconds), which is irreducible given the 60-second minimum analysis timeout required to observe deferred ransomware triggers. The 180-second P99 NFR was met in all test runs.

6.3.4 False Positive Rate Analysis

The false positive suppression engine was evaluated over a 28-day continuous monitoring period on VM3, during which 14 legitimate events that would have triggered raw canary alerts were generated:

- ClamAV on-access scanner scanning canary files: 6 events → suppressed by Tier 2 — AV process whitelist
- rsync backup job accessing /home/user/Documents/: 5 events → suppressed by Tier 1 — Backup software whitelist
- locate/mlocate database update scanning all files: 2 events → suppressed by Tier 3 — Process name whitelist
- Manual file manager browse during maintenance window: 1 event → suppressed by Tier 5 — Time-of-day schedule

Result: 0 false positive alerts generated over the 28-day evaluation period (14/14 legitimate events correctly suppressed). False positive rate: 0.0%. This result validates the effectiveness of the five-tier suppression architecture in eliminating operationally irrelevant events while maintaining detection sensitivity.

6.3.5 Implementation Challenges and Solutions

Table 6.11: Implementation Challenges and Resolutions

Challenge	Root Cause	Solution Implemented	Outcome
CAPEv2 API rate limits under concurrent submissions	Multiple simultaneous alerts from one endpoint generating >5 concurrent CAPE task submissions caused HTTP 429 errors	Added per-CAPE-instance semaphore (max 3 concurrent tasks) in CAPEv2Connector; excess tasks queued in Redis with configurable priority	Zero 429 errors in all subsequent test runs
watchdog inotify IN_ACCESS events flooding	ClamAV on-access scanner generated thousands of IN_ACCESS events per minute during a full disk scan	Implemented debounce filter (500 ms window) on IN_ACCESS events and added ClamAV process whitelist as Tier 2 suppression	CPU spike from 0.7% to 0.1% after debounce; no false alerts
VirusTotal public API quota exhaustion	90 test runs in a single day exceeded the 500 req/day public API quota	Added Redis-backed daily quota counter; when quota >450/500, switch to file-hash-only mode (no IOC enrichment)	Never exceeded quota in production testing schedule
SQLite local buffer corruption on agent crash	Agent crash during an alert write left SQLite WAL file in	Replaced SQLite with a rotating log file (JSON lines format) as the local buffer; added file lock validation on startup	Zero buffer corruption events in 30 forced-crash recovery tests

	inconsistent state, preventing restart		
WebSocket disconnection under heavy alert load	Rapid alert burst (>20 alerts/second from simulated mass deployment) caused Flask-SocketIO to drop events	Added Redis pub/sub as WebSocket event buffer; SocketIO consumers read from Redis rather than direct function calls	No event drops observed at 50 alerts/second sustained load
Windows canary file deletion by test ransomware before agent can restore	Test ransomware deleted and encrypted files in <200 ms, faster than the 500 ms restoration cycle	Added Windows mandatory lock (LOCK_EX via CreateFile with FILE_FLAG_NO_BUFFERING) on canary files to prevent deletion without agent awareness	Ransomware blocked from deleting locked canaries; falls back to encryption of unlocked files, triggering alert earlier

6.3.6 Lessons Learned

The following lessons learned are documented for the benefit of future development iterations and for researchers building upon this work:

- Lesson 1: Async-first design is non-negotiable for security pipelines — Early synchronous sandbox integration (Iteration 3) proved completely infeasible under even minimal concurrent load. The Celery task graph pattern, while adding deployment complexity, is the only viable approach for a pipeline where individual tasks may take 60–300 seconds.
- Lesson 2: Suppression must be designed before deployment, not retrofitted — The first three test runs generated hundreds of false positive alerts from antivirus scanners and backup jobs. The suppression engine was originally a single-tier process whitelist but required rapid expansion to five tiers. Designing suppression architecture before the first canary deployment would have saved significant debugging time.
- Lesson 3: Canary plausibility requires continuous maintenance — During a red team exercise conducted by the project supervisor, the red team identified canary files within 2 minutes by observing uniform file sizes and identical mtime patterns. This motivated the file size randomisation and the canary rotation scheduler described in Section 6.2.5.
- Lesson 4: Certificate management is underestimated — Implementing mutual TLS with a custom CA required significantly more engineering effort than anticipated, including handling certificate renewal, distribution to new agents, and revocation. A dedicated PKI management tool (e.g., CFSSL or Vault) is recommended for production deployments.
- Lesson 5: Performance budgets must be monitored continuously — Agent CPU overhead exceeded the 2% target on three occasions during development due to uncontrolled notify event floods. Integrating psutil self-monitoring into the agent's supervisor loop and adding debounce logic addressed this systematically.

Chapter 7 - Testing

Software Test Plan: IEEE 829 Standard

7.1 Introduction and Test Objectives

This Software Test Plan (STP) is prepared in conformance with the IEEE 829-2008 Standard for Software and System Test Documentation. It governs all validation activities conducted against the Cerberus Multi-Threat Deception and Analysis Platform following the completion of the implementation phase described in Chapter 6.

7.1.1 Purpose

The purpose of this test plan is to document the strategy, scope, procedures, and expected outcomes for the verification of all functional requirements (FRs) and non-functional requirements (NFRs) specified in Chapter 4. Testing confirms that the integrated Cerberus platform: (i) correctly detects canary deception file access across all threat categories; (ii) reliably orchestrates automated submission to the CAPEv2 behavioral analysis engine; (iii) executes endpoint containment and network isolation upon confirmed threats; (iv) enforces all security controls including TLS encryption and JWT access control; and (v) meets the quantified performance constraints in Table 1.5.

7.1.2 Scope

Testing encompasses all six major subsystems: the EndpointAgent, CanaryMonitor, FileMonitor, ProcessMonitor, CerberusComm, and the Central Server's alert ingest, sandbox orchestration, VirusTotal enrichment, and dashboard modules. Integration tests verify end-to-end pipeline behavior from canary file access through to SOC dashboard alert notification and automated response action.

7.2 Test Items and Features to be Tested

The following platform capabilities are subject to formal verification: Canary File Deployment; Canary Integrity Monitoring; Threat Classification; Process Attribution; False-Positive Suppression (all five layers); Sensitive Directory Monitoring; Behavioral Process Detection; Forensic Artifact Collection; Encrypted Alert Transmission; Server-Side Alert Ingestion and Deduplication; CAPEv2 Sample Submission and Tracking; IOC Extraction and MITRE Mapping; VirusTotal Enrichment (Phase 1 and Phase 2); Automated Containment Response; SOC Dashboard Authentication (all three mechanisms); Dashboard Real-Time WebSocket Updates (new_alert, sandbox_complete, vt_file_complete, alert_repeat); Agent Auto-Recovery; Data Retention; and Audit Log Integrity.

7.3 Test Matrix (IEEE 829 Format)

Test ID	Module	Feature Under Test	Expected Outcome	Pass/Fail
TC-001	canary_module.py	Canary deployment – Linux	All 5 canary files created; SHA-256 digests recorded; mtime backdated 365 days	Pass
TC-002	canary_module.py	Canary deployment – Windows	Canary files created in Desktop, Documents, Downloads, C:\Users\Public\Documents	Pass
TC-004	file_monitor.py	Ransomware bait – modified event	on_modified triggers; threat classified as ransomware; severity critical	Pass
TC-005	file_monitor.py	Ransomware bait – deleted event	on_deleted triggers; canary restoration attempted by restore_canary()	Pass
TC-006	file_monitor.py	Credential bait – read event (watchdog	on_opened fires (not on_accessed); threat classified as credential_theft	Pass

		4.0.1)		
TC-007	file_monitor.py	Data stealer bait – access	on_opened fires; threat classified as data_stealer	Pass
TC-008	file_monitor.py	Whitelisted process suppression	No alert generated; event silently dropped	Pass
TC-009	file_monitor.py	Restore blackout (self-trigger suppression)	Agent's own restore write suppressed by RESTORE_BLACKOUT_SECONDS=10	Pass
TC-010	file_monitor.py	Burst suppression	After 3rd alert, _suppressed_until set for 300 s; subsequent alerts dropped	Pass
TC-011	file_monitor.py	Alert cooldown	Second alert within 30 s suppressed; third after 30+ s fires normally	Pass
TC-012	file_monitor.py	Sensitive dir – /etc/passwd modification	on_modified fires; threat credential_theft, severity critical	Pass
TC-015	process_monitor.py	Reverse shell detection (bash one-liner)	REVERSE_SHELL_REGEXES matches /dev/tcp/; threat lateral_movement, severity critical	Pass
TC-016	process_monitor.py	Reverse shell detection (python socket)	REVERSE_SHELL_REGEXES matches socket.*subprocess; threat lateral_movement	Pass
TC-020	forensics.py	Full forensic bundle – live process	memory_dump, process_tree.json, network_connections.json, loaded_modules.json all written	Pass
TC-022	forensics.py	Memory dump – Linux /proc mechanism	/proc/[pid]/mem opened; readable regions accumulated; dump ≥ 1 byte	Pass
TC-023	communicator.py	TLS 1.2+ enforcement (NFR-SEC-001)	All captures show TLSv1.2 or TLSv1.3; downgrade attempt rejected	Pass
TC-025	communicator.py	Invalid API token rejection	Server returns 401 Unauthorized	Pass
TC-026	api/alerts.py	Alert ingestion – new alert creation	Alert, Incident, SandboxJob created; CAPEv2 task dispatched; VT task dispatched; new_alert WebSocket emitted	Pass
TC-027	api/alerts.py	Server-side Redis deduplication	Second POST returns HTTP 200 deduplicated; alert_count incremented; alert_repeat event emitted	Pass
TC-028	api/alerts.py	Server-side whitelist suppression	Returns HTTP 202 suppressed; AuditLog entry created; no Alert record	Pass
TC-030	cape_connector.py	CAPEv2 submission –	MZ magic bytes detected → platform=windows; valid task_id	Pass

		PE binary	returned	
TC-031	cape_connector.py	CAPEv2 submission – ELF binary	\x7fELF magic bytes detected → platform=linux; valid task_id returned	Pass
TC-034	cape_connector.py	Report slimming	Result JSON ≤ 250 KB; top 50 signatures/hosts/HTTP entries retained	Pass
TC-037	cape_connector.py	MITRE ATT&CK mapping	Sorted deduplicated technique IDs (e.g., T1055, T1071, T1486) returned	Pass
TC-038	tasks.py	End-to-end Celery pipeline	Job completes; cape_report, threat_score, malware_family populated; IOCs created; sandbox_complete event emitted	Pass
TC-039	tasks.py	Auto-isolation on critical score	CAPEv2 score ≥ 9: Endpoint.is_isolated=True; next heartbeat response contains action: isolate	Pass
TC-040	vt_connector.py	Phase 1 VT hash lookup	SandboxJob.vt_score populated; vt_file_complete WebSocket event emitted	Pass
TC-041	vt_connector.py	Phase 2 VT IOC enrichment	IOCs receive vt_score, vt_permalink, updated confidence; vt_ioc_complete event emitted	Pass
TC-043	response.py	Linux network isolation (iptables)	INPUT/OUTPUT/FORWARD set to DROP; ACCEPT rules for Cerberus Server IP; host isolated	Pass
TC-044	response.py	Windows network isolation (netsh)	netsh advfirewall blockinbound,blockoutbound; allow rules for server IP	Pass
TC-047	api/auth.py	JWT login – valid credentials	HTTP 200; access_token returned; AuditLog entry result: success	Pass
TC-050	cerberus_agent_service.py	Agent auto-recovery (NFR-REL-001)	Agent restarted by systemd/SCM within 90 seconds of SIGKILL	Pass
TC-053	api/alerts.py	Alert storage capacity (NFR-SCAL)	10,000 Alert records queried; response ≤ 2 seconds	Pass
TC-055	Dashboard (run.py)	Real-time WebSocket new_alert	new_alert event received in browser within ≤ 3 seconds; alert card appears without page refresh	Pass

7.4 Test Deliverables and Environmental Requirements

7.4.1 Test Deliverables

Test artifacts produced and preserved: agent_run.log and agent.log (DEBUG level); Flask and Celery worker server logs; CAPEv2 task JSON reports; PostgreSQL database snapshots (pg_dump) after each integration test; Redis deduplication state dumps (KEYS alert_sig:); Wireshark PCAP captures confirming TLS 1.2/1.3; forensic evidence bundles (memory_dump, process_tree.json, metadata.json); PerformanceMonitor metrics reports over 60-minute windows; and iptables/netsh rule dumps from TC-043 and TC-044.

7.4.2 Environmental Requirements

VM1 (Cerberus Server + CAPEv2): Ubuntu 22.04 LTS; PostgreSQL 13+; Redis 6.2+; CAPEv2 with at least one available guest VM; Flask application and Celery worker running; TLS certificate at /opt/cerberus/certs/server.crt.

VM3 (Linux Agent): Ubuntu 20.04/22.04; agent running as root; watchdog 4.0.1, psutil 5.9.8, requests 2.31.0, cryptography 42.0.5, python-docx 1.1.2 installed.

VM2 (Windows Agent): Windows 10 x64; agent installed as Windows service; procdump.exe at C:\Tools\procdump.exe; Windows Firewall in default configuration.

7.4.3 Pass/Fail Criteria

A test case is passed when the expected outcome is fully satisfied without error, exception, or timeout, as evidenced by the corresponding log or artifact deliverable. All failed test cases require root cause analysis, code correction, and re-execution before the platform is considered ready for final demonstration.

Chapter 8 - Conclusion and Future Work

8.1 Conclusion

This thesis has presented the design, implementation, and evaluation of Cerberus, an advanced Multi-Threat Deception and Analysis Platform developed under the supervision of Dr. Mohamed Hussien at the Arab Academy for Science, Technology and Maritime Transport (AASTMT). The research was motivated by three critical deficiencies observed in contemporary enterprise security operations: the reactive posture of signature-based endpoint detection, the manual bottlenecks inherent in sandbox analysis workflows, and the operational fragmentation arising from disconnected deception, analysis, and response point solutions.

The final implementation demonstrates that the integration of deception-based early-warning with automated dynamic analysis is both theoretically sound and practically achievable within academic resource constraints. The five-module endpoint agent deploys five semantically distinct canary archetypes across monitored directories and detects unauthorized interaction within 1–3 seconds. The CAPEv2-powered analysis pipeline transitions suspicious artifacts from detection to behavioral characterization and MITRE technique mapping within 3–7 minutes — a reduction from the 30–120 minute manual baseline documented in the related work analysis.

A notable architectural contribution is the dual-OS sample routing capability of the CAPEv2Connector, which automatically directs PE binaries to a Windows analysis guest and ELF or shell script samples to a Linux guest based on magic-byte inspection. The VirusTotal v3 enrichment layer, implemented as a two-phase Celery task pipeline with Redis result caching, provides reputation intelligence for both sample hashes and network IOCs, augmenting the platform's threat intelligence output beyond CAPEv2 alone. The layered five-mechanism false-positive suppression architecture ensures the SOC incident queue is populated exclusively with high-confidence, actionable detections. Cerberus demonstrates that an open-source, cost-free implementation of deception-integrated threat detection and automated response — previously available only through commercial products priced at \$50,000–\$500,000 per year — is achievable by a dedicated student engineering team, producing a platform that is both academically novel and operationally viable for small-to-medium organizations and research environments.

8.2 Future Work

8.2.1 Kernel-Level Monitoring via eBPF

The current implementation uses watchdog's user-space inotify/ReadDirectoryChangesW interface, which is vulnerable to kernel rootkits that suppress inotify notifications. A high-priority enhancement is the replacement with an eBPF program attached to `security_file_open`, `security_inode_rename`, and `vfs_write` kernel hooks, delivering canary access notifications at the syscall boundary before any user-space evasion code can suppress them. On Windows, an equivalent minifilter kernel driver operating at `IRP_MJ_CREATE` and `IRP_MJ_WRITE` callback altitudes would provide the same guarantee.

8.2.2 Adaptive and ML-Driven Canary Placement

The current canary deployment strategy is static. A significant improvement would be an adaptive deception engine consuming STIX 2.1 intelligence feeds (via TAXII) and automatically instructing agents to increase canary density in directories corresponding to currently active TTPs. Machine learning models trained on past access-event timelines could further optimize placement to maximize detection probability while minimizing canary file count. Canary content could also embed web beacon URLs (analogous to Thinkst Canarytokens) enabling external access tracking.

8.2.3 Advanced Sandbox Evasion Counter-Measures

Future work should implement a systematic anti-evasion configuration layer within the CAPEv2Connector: CPU core count spoofing to match production endpoint profiles; system artifact enrichment (browser history, document files, registry artifacts); network simulation providing plausible external connectivity; and human interaction injection (mouse movement, window focus events) via CAPEv2 options parameters.

8.2.4 Enterprise SIEM Integration via STIX/TAXII and CEF Streaming

Future work should implement a fully conformant STIX 2.1 export interface serializing completed incidents as STIX Bundles published to a TAXII 2.1 Collection server endpoint, enabling direct consumption by commercial SIEM platforms (Splunk, Microsoft Sentinel, IBM QRadar). In parallel, a Common Event Format (CEF) syslog streaming

module should translate each Alert and AuditLog record into a CEF-formatted message transmitted to the SIEM's syslog receiver.

8.2.5 Distributed Multi-Server Architecture

The current single-server deployment creates a single point of failure. Future work should introduce a distributed architecture: load-balanced Flask API tier (Nginx reverse proxy), horizontally scalable Celery workers, Redis Cluster for broker high-availability, and PostgreSQL primary-replica pairs with Patroni failover. Kubernetes orchestration would enable automated scheduling, health checking, and rolling updates, targeting the extended NFR-SCAL-001 goal of 500+ concurrent monitored endpoints.

8.2.6 Memory Forensics with Volatility 3 Integration

The current forensic collection reads raw memory from `/proc/[pid]/mem`, producing a binary dump requiring manual analysis. A high-value enhancement would integrate Volatility 3 into the analysis pipeline: after CAPEv2 completes, the persisted memory dump would be submitted to a Volatility analysis job running plugins (`linux.pslist`, `linux.malfind`, `windows.cmdline`) to automatically extract process artifacts and injected memory regions. The JSON-serializable Volatility 3 output would be ingested as additional IOC records with `ioc_type = "memory_artifact"`, enriching the incident timeline with evidence complementary to the CAPEv2 behavioral report

References

References (with Links)

1. Thinkst Applied Research. (2023). *Canarytokens: Practical Deployment of Honeytokens at Scale*.
<https://thinkst.com/research.html>
<https://canarytokens.org/generate>
2. Spitzner, L. (2003). *Honeypots: Tracking Hackers*. Addison-Wesley.
<https://www.pearson.com/en-us/subject-catalog/p/honeypots-tracking-hackers/P200000006295>
3. Bowen, B. M., Hershkop, S., Keromytis, A. D., & Stolfo, S. J. (2009). Baiting inside attackers using decoy documents. *SecureComm*.
https://link.springer.com/chapter/10.1007/978-3-642-04444-1_4
4. Smith, J., & Johnson, M. (2019). HoneyFiles: Deceptive files for intrusion detection. *ACM CCS*.
https://www.researchgate.net/publication/336909897_HoneyFiles_Deceptive_Files_for_Intrusion_Detection
5. Kharraz, A., Robertson, W., Balzarotti, D., Bilge, L., & Kirda, E. (2016). UNVEIL: A large-scale, automated approach to detecting ransomware. *USENIX Security*.
https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_kharraz.pdf
6. Egele, M., Scholte, T., Kirda, E., & Kruegel, C. (2012). A survey on automated dynamic malware analysis techniques and tools. *ACM Computing Surveys*.
https://www.researchgate.net/publication/334416385_A_survey_on_automated_dynamic_malware-analysis_techniques_and_tools
7. Continella, A., et al. (2016). ShieldFS: A self-healing, ransomware-aware file system. *ACSAC*.
<https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/continella>
8. Scaife, N., Carter, H., Traynor, P., & Butler, K. R. B. (2016). CryptoLock (and drop it): Stopping ransomware attacks on user data. *IEEE ICDCS*.
<https://www.ndss-symposium.org/wp-content/uploads/2017/09/11-3-1.pdf>
9. Chen, K., & Zhang, W. (2021). A multi-layer deception framework for ransomware detection. *Computers & Security*, 105.
<https://www.sciencedirect.com/science/article/pii/S0167404821000847>
10. McIntosh, T., Jang-Jaccard, J., & Watters, P. (2019). Early detection of ransomware using honeypots and machine learning. *IEEE*.
https://www.researchgate.net/publication/332333556_Early_Detection_of_Ransomware_using_Honeypots_and_Machine_Learning
11. National Institute of Standards and Technology. (2023). *Computer Security Incident Handling Guide (SP 800-61 Rev. 2)*.
<https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final>
12. Verizon. (2024). *2024 Data Breach Investigations Report*.
<https://www.verizon.com/business/resources/reports/dbir/>
13. IBM Security. (2024). *Cost of a Data Breach Report 2024*.
<https://www.ibm.com/reports/data-breach>
14. FireEye (Mandiant). (2024). *M-Trends 2024*.
<https://www.mandiant.com/resources/reports/m-trends>
15. AV-TEST Institute. (2024). *Malware Statistics*.
<https://www.av-test.org/en/statistics/malware/>
16. Gartner. (2024). *Market Guide for Deception Technology*.
<https://www.gartner.com/en/reviews/market/deception-technology>

17. SANS Institute. (2024). *Incident Response Survey*.
<https://www.sans.org/white-papers/39233/>
18. Enterprise Strategy Group. (2024). *SOC Modernization and Automation*.
<https://www.esg-global.com/research>
19. CAPEv2 Project. (2024). *CAPEv2 Documentation*.
<https://github.com/kevoreilly/CAPEv2>
<https://capev2.readthedocs.io/en/latest/>
20. Python Software Foundation. (2024). *Python 3.12 Documentation*.
<https://docs.python.org/3/>
21. MITRE Corporation. (2024). *MITRE ATT&CK Framework*.
<https://attack.mitre.org/>
22. IEEE. (1998). *IEEE 830-1998 Software Requirements Specification*.
<https://standards.ieee.org/ieee/830/1229/>
23. NIST. (2023). *Cybersecurity Framework 2.0*.
<https://www.nist.gov/cyberframework>
24. OWASP. (2024). *OWASP Top Ten 2024*.
<https://owasp.org/www-project-top-ten/>
25. University of California, Santa Barbara. (2018). *DECAF Framework*.
<https://www.sysnet.ucsb.edu/projects/decaf>
26. Carnegie Mellon University. (2020). *ShadowNet Deception Research*.
<https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=533496>
27. Georgia Institute of Technology. (2020). *Malware Analysis Workbench*.
<https://www.cc.gatech.edu/security-publications>
28. YARA Rules Project. (2024). *YARA Rules Repository*.
<https://github.com/Yara-Rules/rules>
<https://yara.readthedocs.io/>
29. VirusTotal. (2024). *VirusTotal Community*.
<https://www.virustotal.com/gui/community>
30. Malware Traffic Analysis. (2024). *Malware Traffic Analysis*.
<https://www.malware-traffic-analysis.net/>

Appendix A - Senior Project Summary Report

Project Title	Cerberus Platform: Multi-Threat Deception and Analysis Platform	
Supervisor(s)	Dr. Mohamed Hussien	
Team members:	Names Abdelrahman Mohamed Mahmoud Mohamed Nour Mohamed Mahmoud Karim Osama Abdelmoniem Mohamed Tamer Eid Mohamed Hossam Mohamed Omar Hesham Hamed	Registration Numbers 221027770 221027637 221027651 221007879 221007946 221027535
Project Deliverables	• <i>A centralized web-based SOC dashboard for real-time visualization and incident management.</i> • <i>Cross-platform endpoint protection agents compatible with Windows and Linux environments.</i> • <i>A Central Management Server utilizing a RESTful API and PostgreSQL database.</i> • <i>Integration with the CAPEv2 Sandbox for automated malware behavioral analysis.</i> • <i>Automated incident response workflows mapping to the MITRE ATT&CK framework.</i> • <i>A deception canary management module for deploying and monitoring honeypot files.</i>	
Local Impact	Economic/Organizations: Provides Small/Medium Businesses (SMBs) and academic institutions with an affordable, enterprise-grade security solution that operates within limited budgets. Operational: Eliminates licensing costs by utilizing open-source foundations, democratizing advanced threat detection for resource-constrained organizations. Individuals: Reduces the workload and alert fatigue for local Security Operations Center (SOC) analysts.	
Global Impact	Research Contribution: Advances global cybersecurity research by providing a validated, open-source, and replicable implementation of an integrated deception and automated analysis architecture. Threat Mitigation: Helps reduce the global median dwell time of attackers by orchestrating automated containment actions within minutes instead of hours.	
Team Organization	The team integrated multiple disciplines including Cybersecurity, Systems Engineering, Software Development, Network Security, and Data Science to address conflicting technical issues.	
Ethical Considerations	• Data Privacy: The platform avoids collecting Personally Identifiable Information (PII), focusing only on monitoring file access patterns rather than content. • Testing Safety: Ethical testing was enforced by sourcing malware samples exclusively from approved academic repositories.	

	• Ethical Use: Clear documentation is provided to ensure the responsible deployment of the deception platform.
Social Impact	• Improves trust in automated security systems by providing detailed forensic context and justification for every automated containment action. • Enhances the overall security posture of organizations, thereby protecting societal data from severe threats like ransomware and data stealers.
Professional Responsibility	• Fail-Safe Design: Engineered the system with conservative automation defaults to prevent catastrophic failures or operational disruptions caused by false positives. • License Compliance: Ensured all utilized third-party software and libraries strictly adhered to their open-source licenses (e.g., Apache 2.0, GPLv3, MIT) with proper attribution.
Standards	• IEEE Standards: IEEE 12207, IEEE 830, IEEE 829. • ISO/IEC Standards: ISO/IEC 25010 (SQuaRE), ISO/IEC 27001. • Security & Threat Frameworks: NIST Cybersecurity Framework, OWASP Top 10, MITRE ATT&CK. • Data & Design Standards: UML 2.5, RESTful API Design, JSON Schema, OpenAPI 3.0, STIX 2.1, Common Event Format (CEF), PEP 8. • Compliance: GDPR.
Realistic Constraints	• Performance: Endpoint agent CPU utilization must not exceed an 8% average, memory footprint must stay below 150MB baseline, and detection latency must remain under 10 seconds. • Security: Agent-server communications required TLS 1.2+ encryption, and stored sensitive data required AES-256 encryption. • Compatibility: Mandatory support for Windows 10/11 (x64) and Ubuntu 20.04+ (x64) environments.

Supervisor Name	Signature